데이터 인덱싱
Indexing (LSM-tree)

Prof. Youngjae Kim

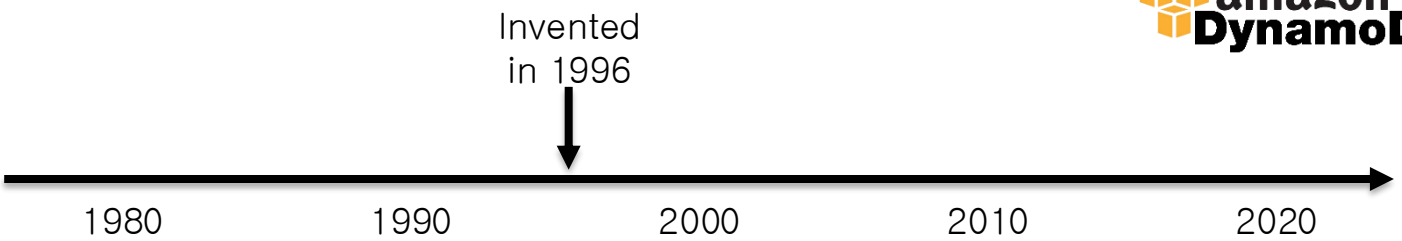
- **Contents covered in this lecture:**
  - What is an LSM-tree data structure?
  - The data structures and algorithms of LSM-trees
  - The approache to resolving RocksDB write stall issues using KVSSDs)

* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Useful When?

- ✓ Massive dataset
- ✓ Rapid updates/insertions
- ✓ Fast lookups

LSM-trees are for you



* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Applications of LSM-tree

- **Key-value Stores (KVS)** has become a necessary infrastructure
- Indexing and Caching (redis & memchached)
  - B-Tree based; Hash based
- Storage Systems (Persistent KVS)
  - NoSQL
    - BigTable, HBase, Cassandra
  - Storage engine
    - LevelDB, RocksDB, MyRocks

* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

NoSQL Database

- BigTable (Google)
  - Column-family-based NoSQL database
  - Built on top of GFS (Google File System)
  - Used in Gmail, Google Analytics, and Google Maps
- HBase (Apache – Open-source implementation of BigTable)
  - Distributed, column-oriented NoSQL database built on the Hadoop ecosystem
  - Uses HDFS (Hadoop Distributed File System) as the storage layer
  - Well integrated with Hadoop tools such as MapReduce, Hive, and Pig
  - Easy to scale
- Cassandra (Originally developed by Facebook and currently maintained by Apache)
  - Peer-to-peer (P2P) architecture-based distributed NoSQL database
  - Unlike HBase, it does not have a master node
  - Strong in replication and fault tolerance, and easy to scale

* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Beginning of LSM-tree

- **1992: Log-Structured File System (LFS)**
  - Out-of-place updates
  - Sequential writes
  - Garbage collection
  - Motivated by improving write throughput on disks
- **1996: Log-Structured Merge Tree (LSM-tree)**
  - Out-of-place updates
  - Optimized for write-intensive workloads
  - Trade-off: faster writes, slower reads
  - Requires periodic data reorganization
    - merge / compaction operations

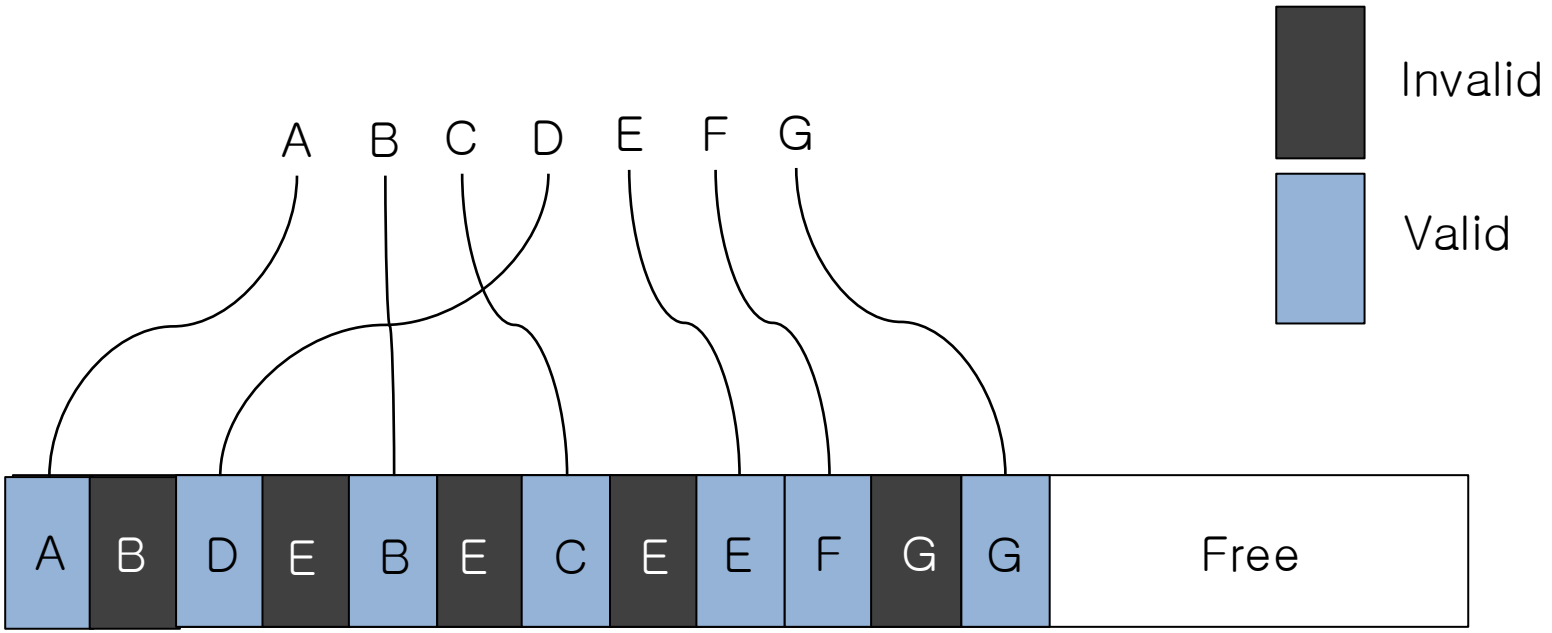
Patrick N'Neil
Umass Boston

* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Log-Structured *

Write Sequence:

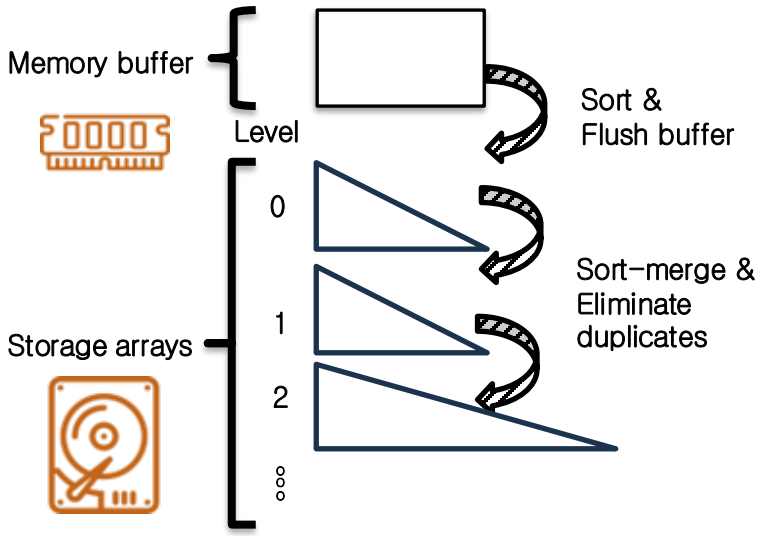
A -> B -> D -> E -> B -> E -> C -> E -> E -> F -> G -> G



* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Basic LSM-tree

- **Architecture Overview**
- **Two main components**
  - Memory component (MemTable / buffer)
  - Storage component (SSTable arrays across levels)
- **Design Principle 1 — Write Optimization**
  - **Buffer writes in memory** to maximize sequential I/O
  - Periodically **sort** and **flush** the buffered data to disk
- **Design Principle 2 — Read Optimization**
  - **Sort–merge** data across levels
    - **Eliminate duplicates** during compaction
    - Maintain **sorted arrays** for efficient lookups

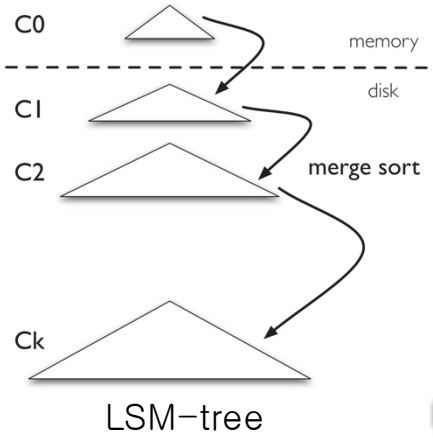


- Memory buffer → Sort & Flush buffer → Level-0 → Level-1 → Level-2 …
- Lower levels contain larger, more compacted sorted arrays
- Merge operations maintain global ordering

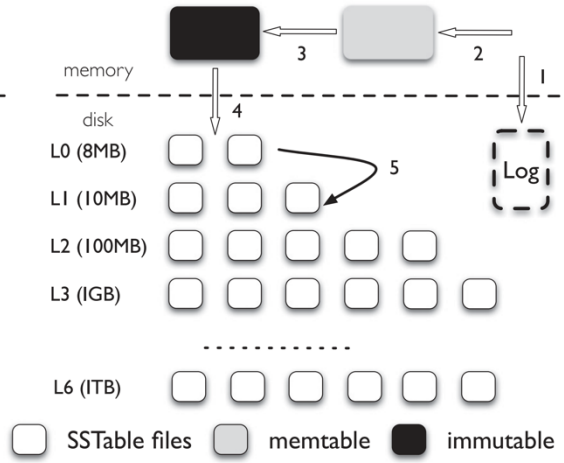
* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Modern Structure of LSM-tree

- Structure
  - Memory component (MemTable)
  - Disk Component (SSTable)
    - Level 0 and other levels
  - Log (WAL/MANIFEST)
- Operations
  - Read (point and range queries
  - Write
    - Memory write
    - Flush
    - Compaction
  - Delete is also write (write tombstone)



LSM-tree

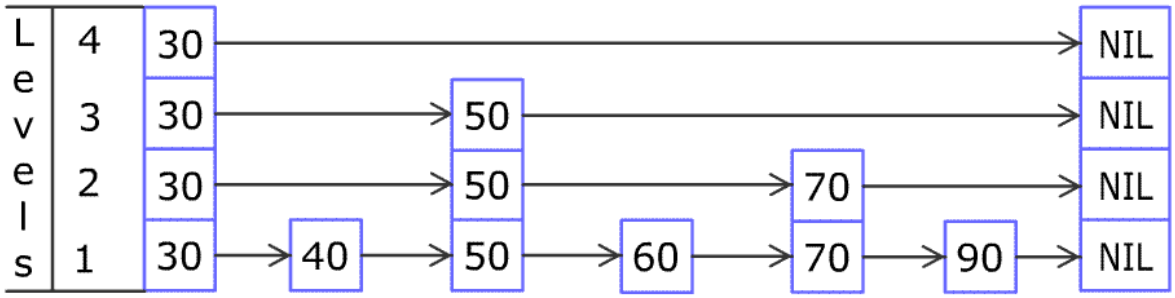


RocksDB

* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Modern Structure of LSM-tree

- Memory Components:
  - Called MemTable
  - Skiplist based in-memory data structure
  - High write and read performance
  - Limited total size
  - Becomes immutable once reaches a size threshold



* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Modern Structure of LSM-tree

- Disk Components
  - Sorted String Table format
  - Divided into logical levels
  - Exponentially increasing capacities

| Data Block 1 | Data Block 2 | ... | Data Block N | Metaindex Block | Index Block | Footer |

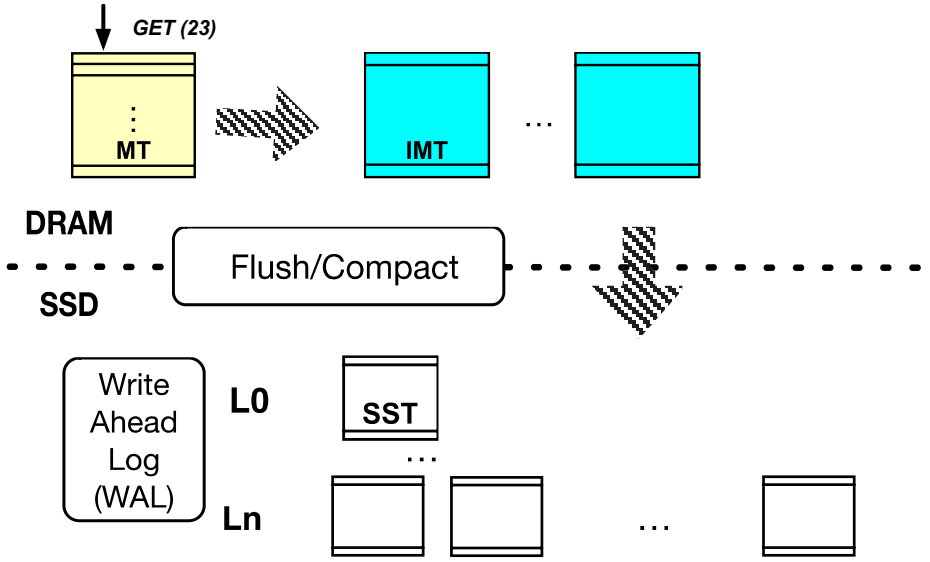
SSTable format (Slot Directory (or Slot Table))

Component	Role
Data Block	Stores the actual key-value pairs
Index Block	Quickly locates the position of Data Blocks
Filter Block	Quickly determines if a key does not exist to reduce I/O
Metaindex Block	Indexes metadata blocks such as the Filter Block
Footer	Stores the locations of the Metaindex Block and Index Block

* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Operations – Point Query

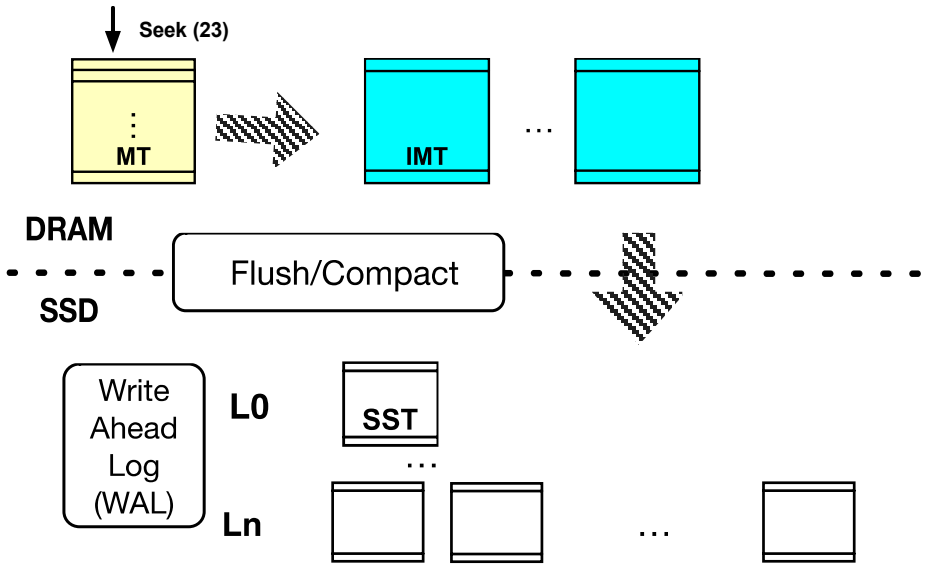
- Returns immediately when something found
- Problems:
  - Read amplification
    - Files: (worst case) search (n -1 + files number of level 0) files.
    - Inside the file
- Optimization:
  - Page cache/Block cache
  - Bloom filter
  - Other index/filter



* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Operations – Range Query

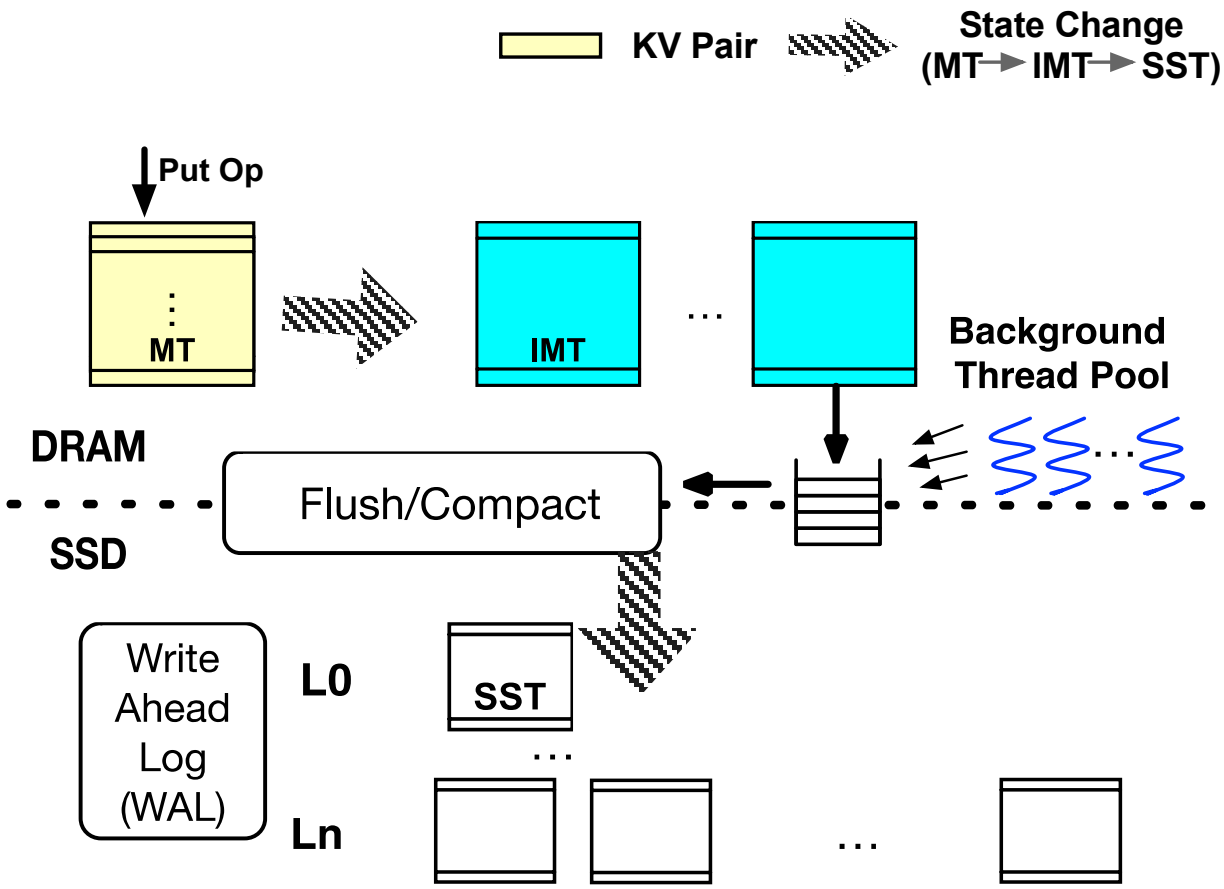
- Must seek every sorted run
- Bloom filter not support range query
- Optimization:
  - Parallel seeks
  - Prefix bloom filter
  - Other index




```
for(itr->Seek(23); itr->Valid();
itr->Next()){
  if(itr->Key() < 40{
      …..
  } else  …..
}
```

* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

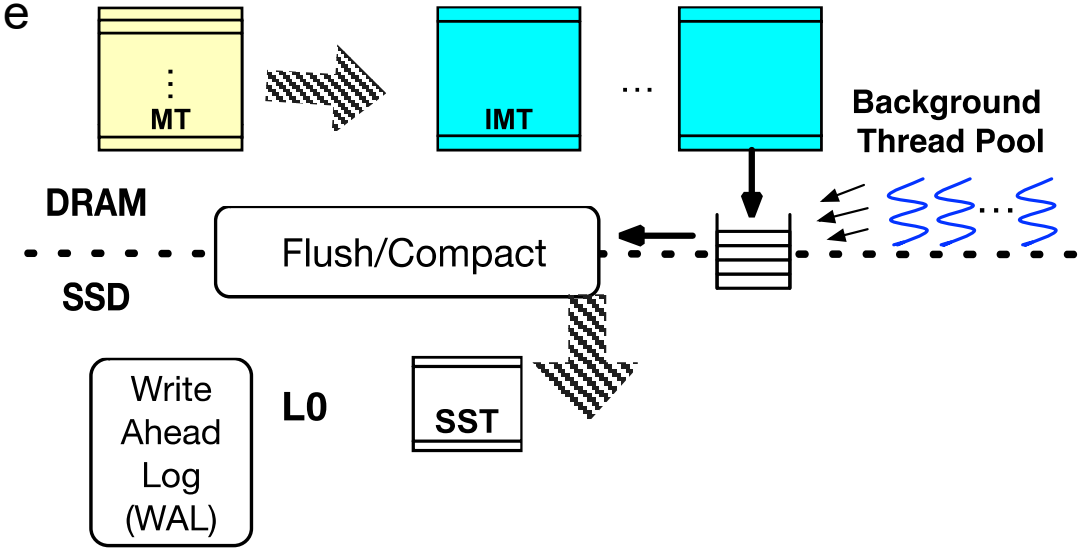
Operations – Write



* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Operations – Flush

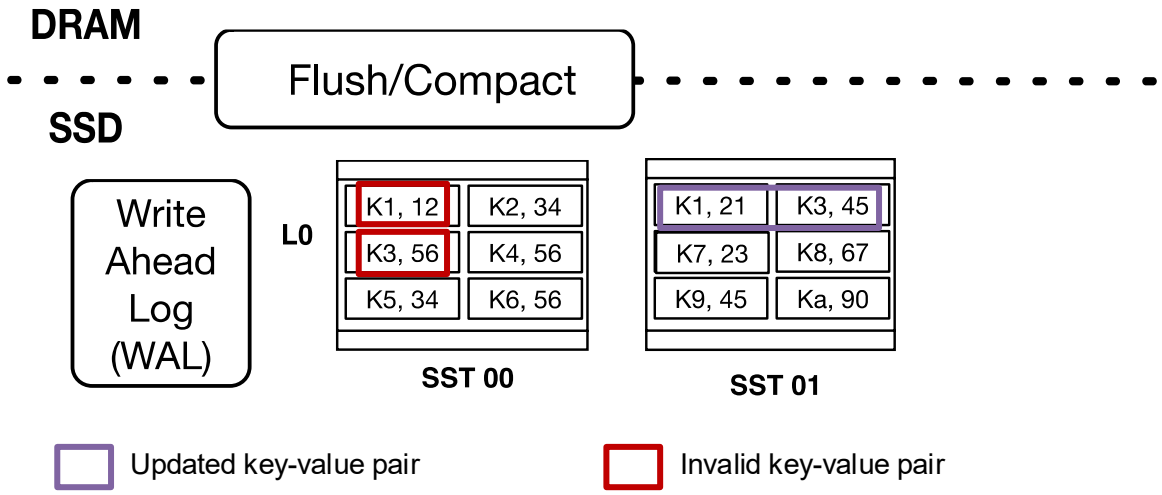
- Internal operations
  - Flush operation – when Immutable MemTable (IMT) is written to storage in SSTable format
    - Mostly performed by background threads
    - One of the performance defining operation
    - Converts IMT to SSTable format
    - Append it to Level 1 of disk



* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Operations - Compaction

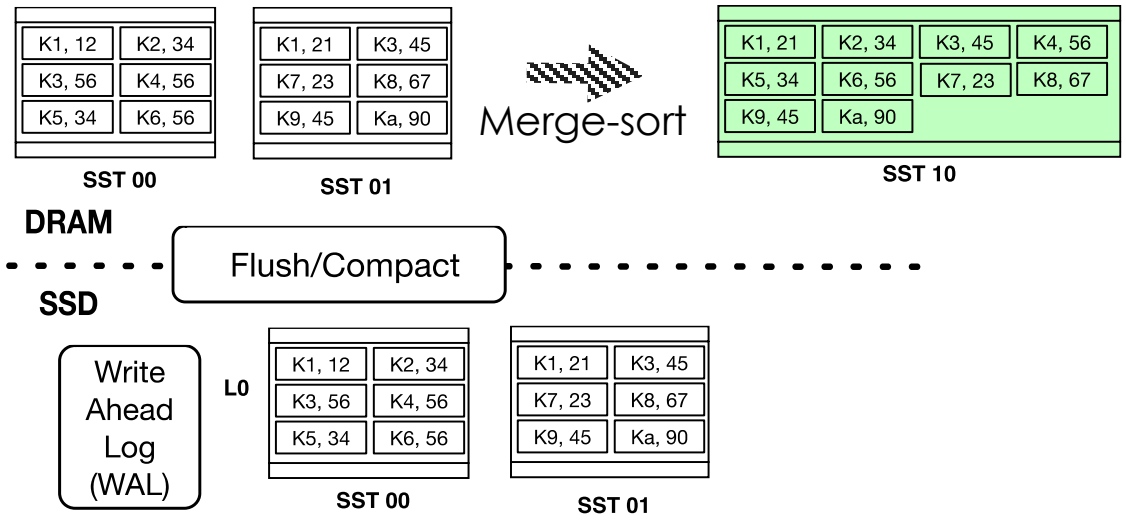
- Internal operations
  - Compaction operation – Sort-merge operation to maintain tree like structure
    - Reclaim invalid data
    - Reduces the number of SSTables in LSM-tree
    - Perform merge-sort operation between SSTables
    - Multiple types of compaction operations



* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Operations - Compaction

- Internal operations
  - Compaction operation – Example



❑ This Merge-Sort operation led to high number of internal writes which we call write amplification

* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

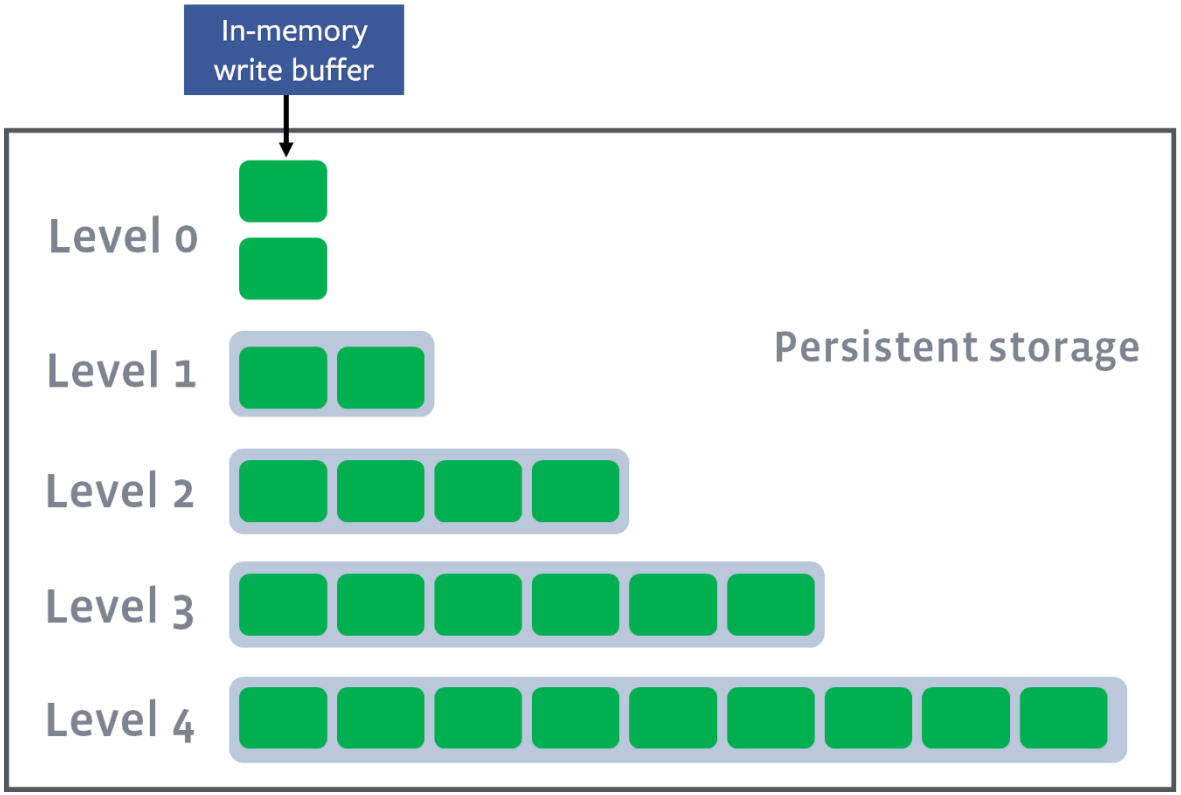
Operations - Compaction

- Tiered compaction
  - Multiple SSTable files are accumulated at the same level (e.g., L0, L1, etc.).
  - When the number of files exceeds a certain threshold (N files), they are merged (compacted) and moved to a lower level (or sometimes remain at the same level).
  - Since compactions happen relatively infrequently, write performance is very high.
  - However, because data is scattered across multiple files, a large number of files need to be scanned during reads, which can result in poor read performance.
- Advantageous when write performance is the top priority.

* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Operations - Compaction

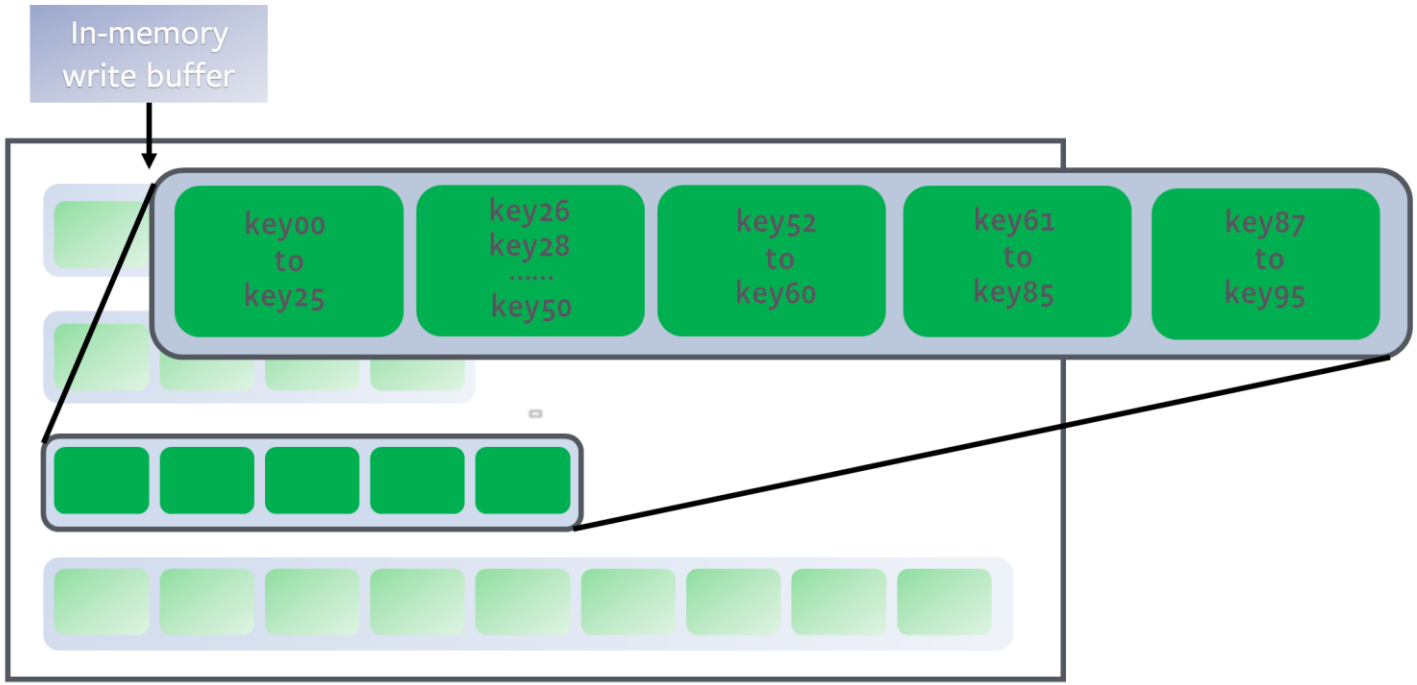
- Leveled compaction
  - Each level manages SSTable files such that their key ranges do not overlap.



* https://github.com/facebook/rocksdb/wiki/Leveled-Compaction
* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Operations - Compaction

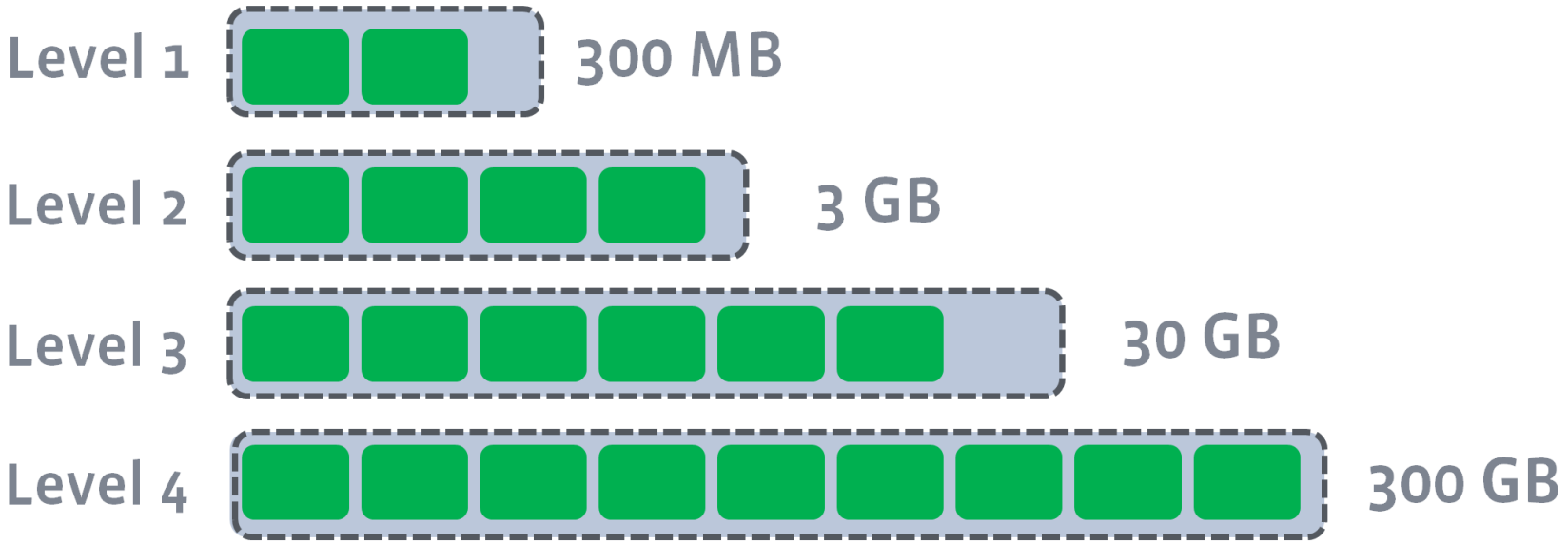
- Leveled compaction
  - Each level manages SSTable files such that their key ranges do not overlap.



* https://github.com/facebook/rocksdb/wiki/Leveled-Compaction
* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Operations - Compaction

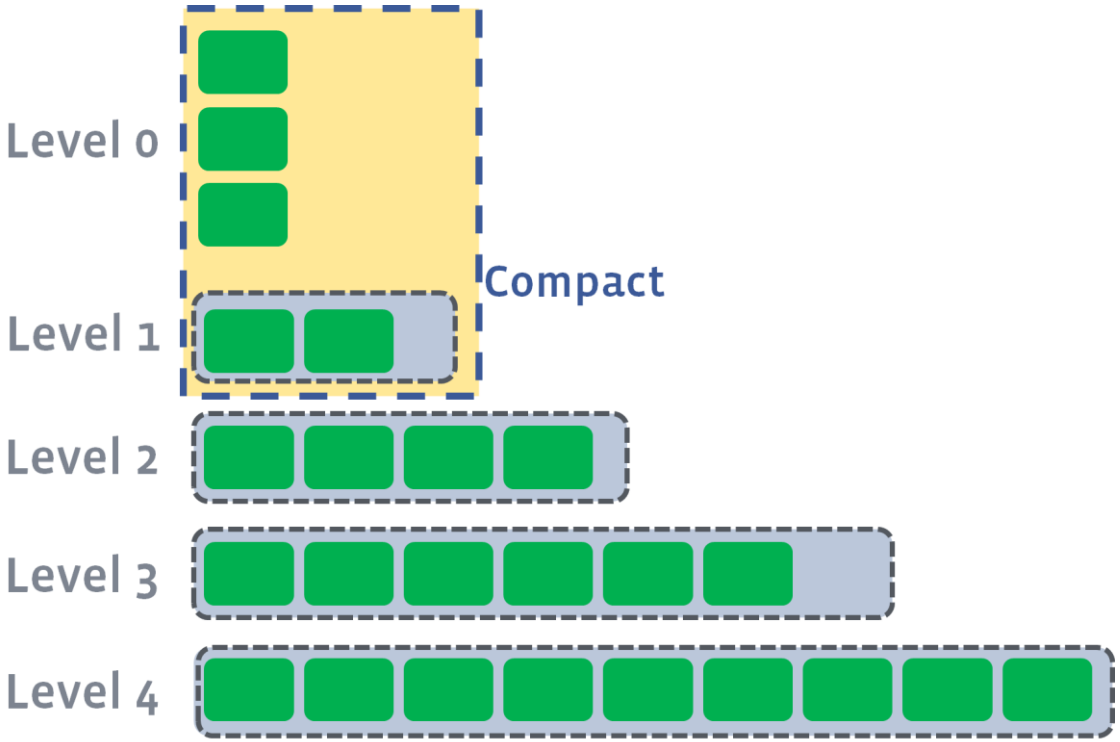
- Leveled compaction
  - As the level increases, the amount of data typically grows by a factor of 10 compared to the previous level (Level N is about 10 times larger than Level N-1).



* https://github.com/facebook/rocksdb/wiki/Leveled-Compaction
* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Operations - Compaction

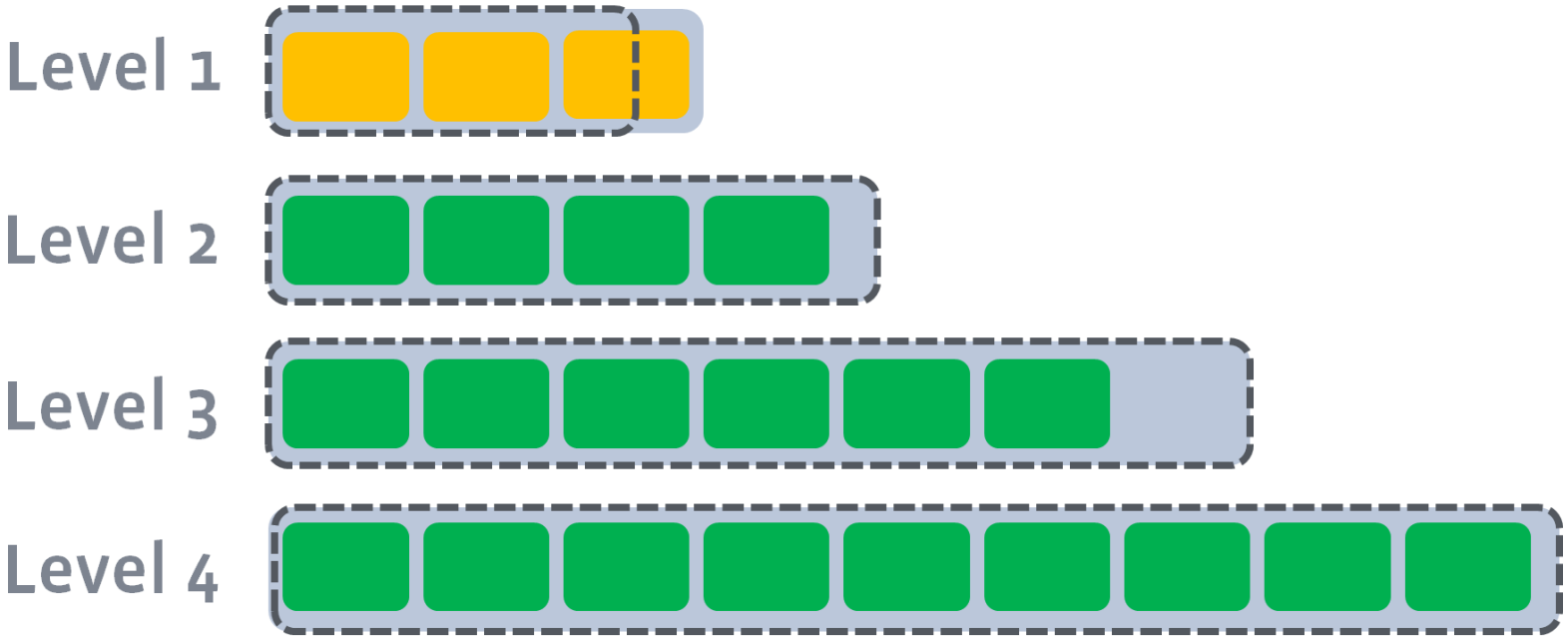
- Leveled compaction
  - Compaction triggers when number of L0 files reaches a certain trigger point, files of L0 will be merged into L1.



* https://github.com/facebook/rocksdb/wiki/Leveled-Compaction
* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Operations - Compaction

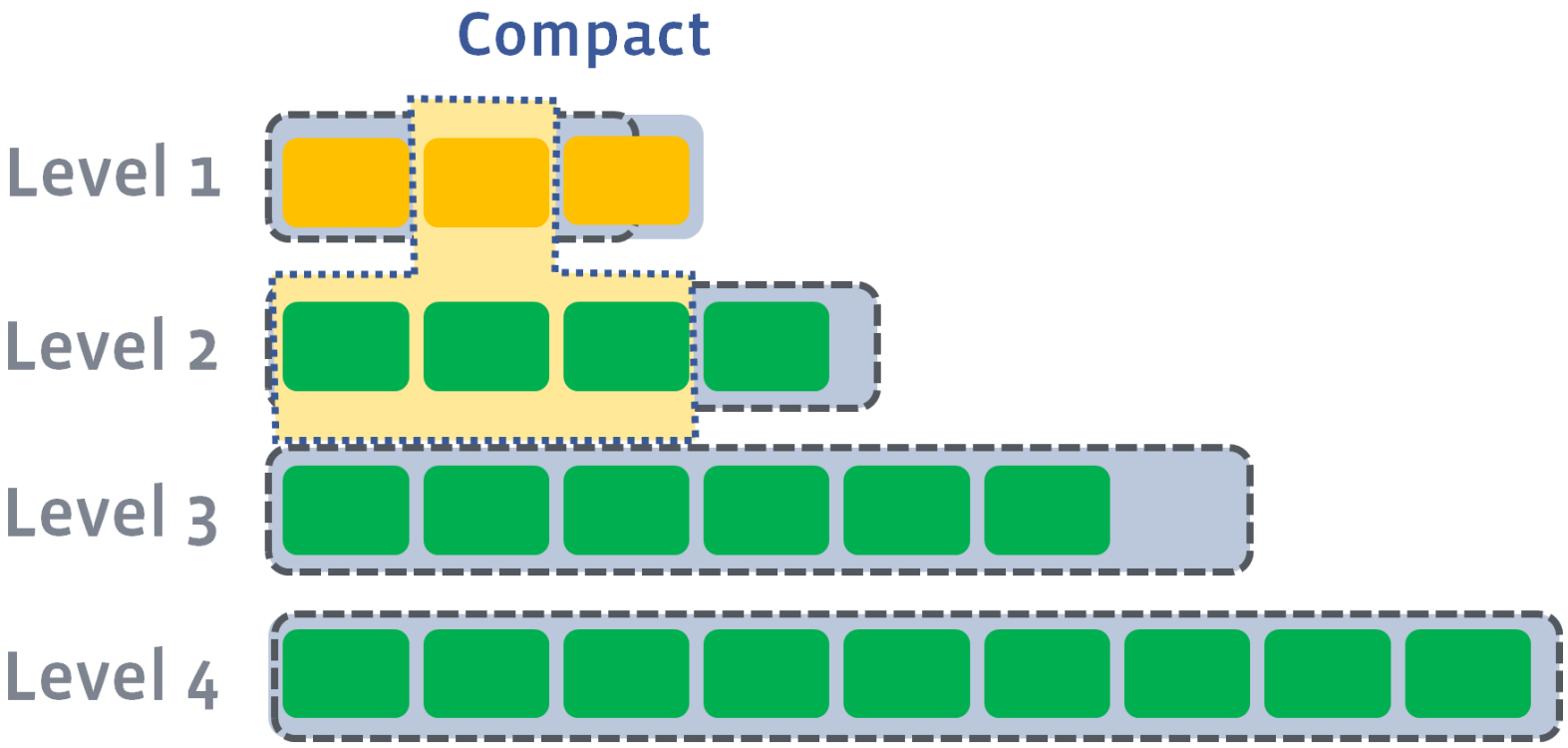
- Leveled compaction
  - After the compaction, it may push the size of L1 to exceed its target.



* https://github.com/facebook/rocksdb/wiki/Leveled-Compaction
* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Operations - Compaction

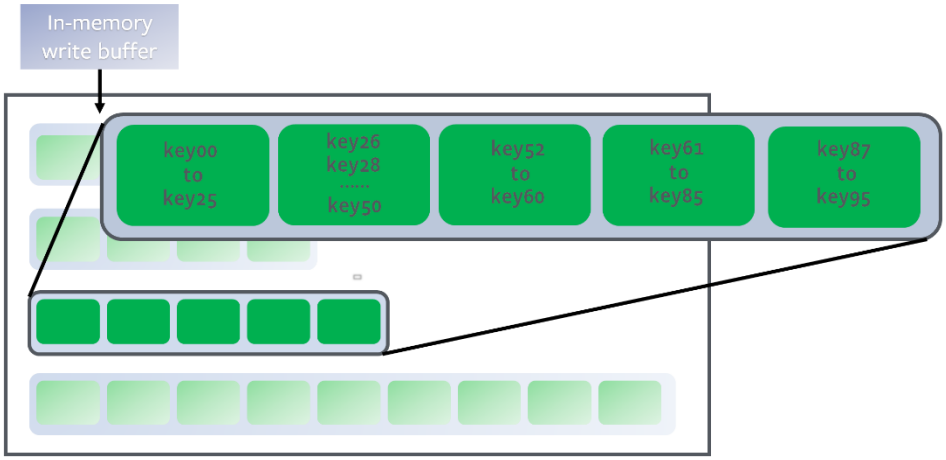
- Leveled compaction
  - Pick at least one file from L1 and merge it with the overlapping range of L2. The result files will be placed in L2.



* https://github.com/facebook/rocksdb/wiki/Leveled-Compaction
* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Operations - Compaction

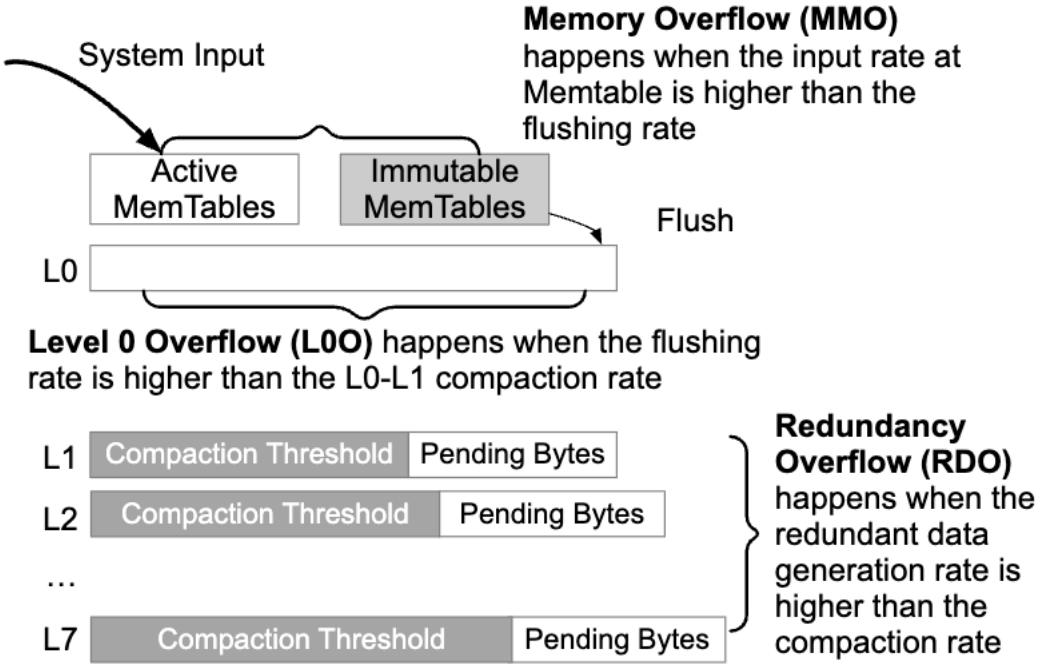
- Leveled compaction
  - During reads, only a single file in a particular level needs to be accessed to find the desired key, resulting in excellent read performance.
  - The downside is that compactions occur very frequently, which can lead to slower write performance due to compaction overhead.
- Advantageous when read performance is the top priority.



* https://github.com/facebook/rocksdb/wiki/Leveled-Compaction
* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Operations – Write Stall

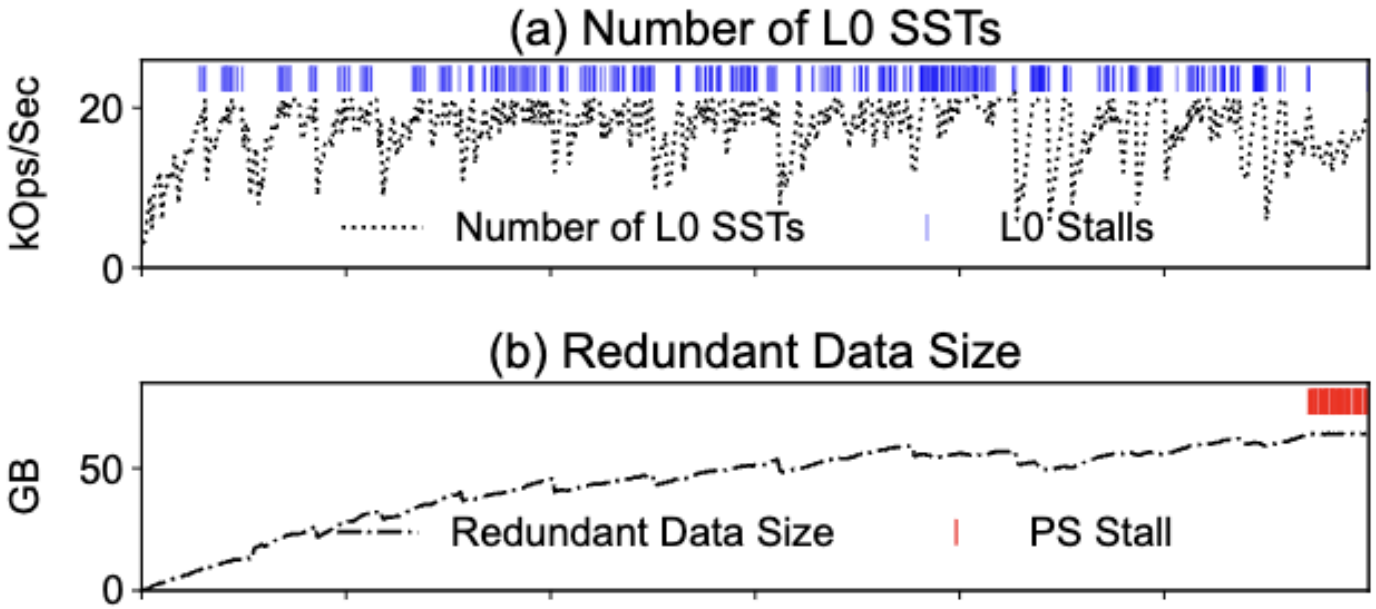
- Write Stall
  - RocksDB introduces *write stalls* to prevent the system from being overwhelmed by write amplification, compaction backlog, and excessive memory or disk usage.



* https://www.usenix.org/system/files/fast23-yu.pdf
* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Operations – Write Stall

- Write Stall
  - If the number of L0 SST file reaches 20, the user write is blocked. (L0 stall)
  - If Redundant Data Size reaches 64GB, the user write is blocked. (Pending stall)
  - If the Memtable is full and not flushed yet, the user write is blocked (Memtable stall)



* https://www.usenix.org/system/files/fast23-yu.pdf
* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

Operations – Write Summary

- Write log firstly, and write in memory (critical path)
- Flush from memory to disk (write stall)
- Compactions (write amplification)
- Optimization
  - Compaction algorithms
  - Client operation and internal operation tradeoff
  - Cache …

* A. Silberschatz et al., *Database System Concepts (7th ed.)*, McGraw-Hill, 2020.

KVAccel: A Novel Write Accelerator for LSM-Tree-Based KV Stores with Host-SSD Collaboration

KiHwan Kim, Hyunsun Chung, Seonghoon Ahn, Junhyeok Park, Safdar Jamil, Hongsu Byun, Myungcheol Lee, Jinchun Choi, Youngjae Kim, **KVAccel: A Novel Write Accelerator for LSM-Tree-Based KV Stores with Host-SSD Collaboration**, IEEE Int'l Parallel and Distributed Processing Symposium (IPDPS) (2025), Milan, Italy, June 3-7, 2025.

KVACCEL: A Novel Write Accelerator for LSM-Tree-Based KV Stores with Host-SSD Collaboration | IEEE Conference Publication | IEEE Xplore

LSM-tree based Key-Value Stores (LSM-KVS)

- Log-Structured Merge-Tree(LSM-tree)
  - Designed for write-intensive workloads
  - Optimized for large-scale data
  - Out-of-place updates
  - Sequential batch operations

RocksDB [1]

levelDB [2]

ZippyDB [3]

[1]: Facebook, "RocksDB" https://rocksdb.org, 2012
[2]: Google, "LevelDB" https://github.com/google/leveldb, 2017
[3]: Meta, "ZippyDB" https://engineering.fb.com/2021/08/06/core-infra/zippydb/, 2021

LSM-tree based Key-Value Stores (LSM-KVS)

- LSM KVS(e.g. RocksDB) stores data in an append-only manner in the active MemTable
- Data in MemTable is moved to and managed on disk through background jobs(Flush, Compaction)

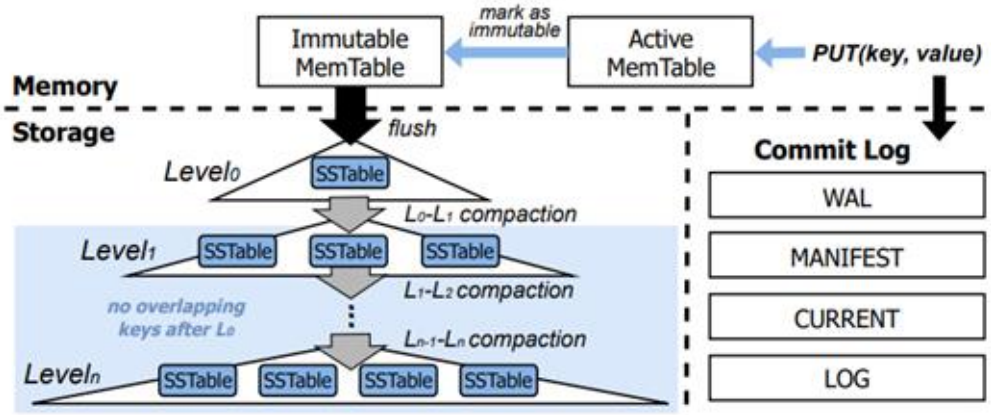


Fig. 1: An architecture of LSM-tree.

Write Stall Problem

- Write Stall: write operation blocked, due to bottlenecks in Flush, Compaction
- In RocksDB, Write stall occurs under these 3 scenarios[4][5]
  - Incoming Writes > Flush
  - Flush > Level 0 to Level 1 Compaction
  - Pending deep level compaction size becomes heavier

[4]: SILK: Preventing Latency Spikes in Log-Structured Merge Key-Value Stores, Oana Balmau et al., USENIX ATC'19
[5]: ADOC: Automatically Harmonizing Dataflow Between Components in Log-Structured Key-Value Stores for Improved Performance, Jinghuan Yu et al. (USENIX FAST'23)

Existing Work: ADOC[5]

- In three types of overflow scenarios, ADOC alleviates write stalls by adjusting two tuning knobs
- Two tuning knobs: # of Compaction threads, MemTable size

	# of Compaction Threads	MemTable Size
Incoming Writes > Flush	↓	↑
Flush > Level 0 to Level 1 Compaction	↑	
Pending deep level compaction size becomes heavier	↑	↓

[5]: ADOC: Automatically Harmonizing Dataflow Between Components in Log-Structured Key-Value Stores for Improved Performance, Jinghuan Yu et al. (USENIX FAST'23)

Existing Work: ADOC[5]

- In three types of overflow scenarios, ADOC alleviates write stalls by adjusting two tuning knobs
- Two tuning knobs: # of Compaction threads, MemTable size

1. Not an immediate remedy → Write stalls still occur

2. Tuning knobs does not stop write slowdowns from occurring.

Pending deep level compaction size becomes heavier	⇧	⇩

[5]: ADOC: Automatically Harmonizing Dataflow Between Components in Log-Structured Key-Value Stores for Improved Performance, Jinghuan Yu et al. (USENIX FAST'23)

Observation 1.
Slowdowns[6]: The Inefficient Write Stall Solution

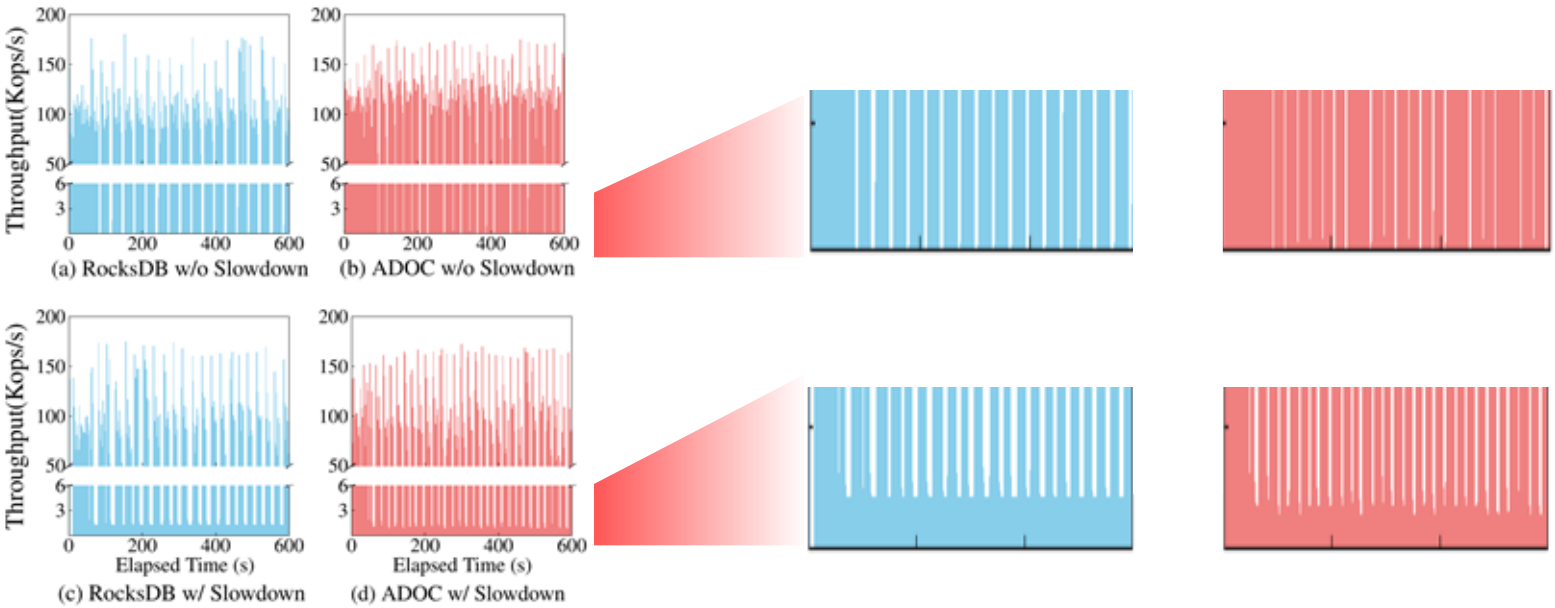
- RocksDB uses the *slowdown*[6] method to prevent user writes from becoming completely blocked
- The state of the art solution ADOC[5] also uses *slowdowns*

➡ Both RocksDB and ADOC[5] ultimately fall back to using *slowdown* to avoid a write stall

[5]: ADOC: Automatically Harmonizing Dataflow Between Components in Log-Structured Key-Value Stores for Improved Performance, Jinghuan Yu et al. (USENIX FAST'23)
[6]: https://github.com/facebook/rocksdb/wiki/Write-Stalls

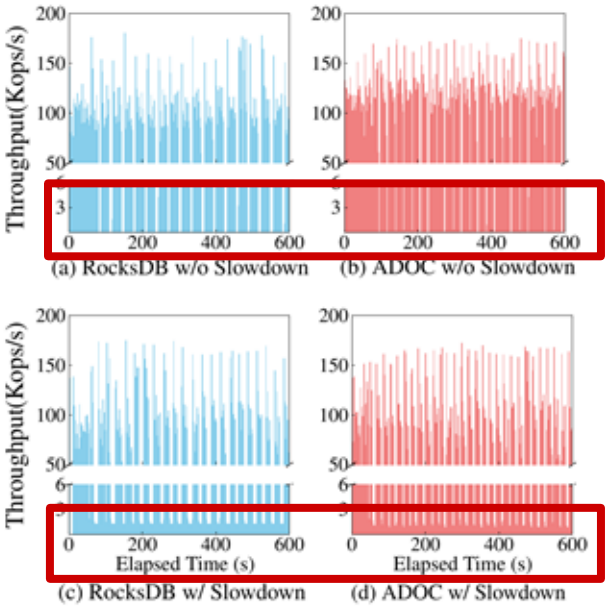
Observation 1.
Slowdowns[6]: The Inefficient Write Stall Solution

- *Slowdowns*, while preventing a complete write stall from occurring, harms overall performance

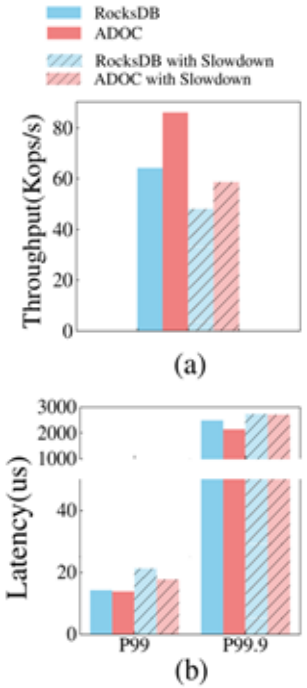


Observation 1.
Slowdowns[6]: The Inefficient Write Stall Solution

- *Slowdowns*, while preventing a complete write stall from occurring, harms overall performance



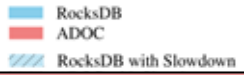
I/O service is uninterrupted thanks to slowdowns preventing write stalls...



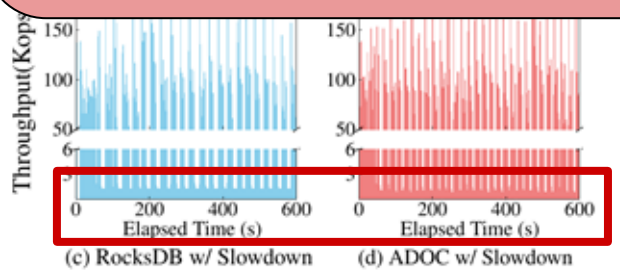
…At the cost of overall throughput and latency

Observation 1.
Slowdowns[6]: The Inefficient Write Stall Solution

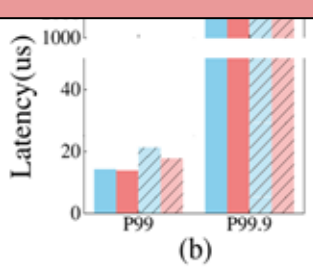
- *Slowdowns*, while preventing a complete write stall from occurring, harms overall performance.



Both state-of-the-art and industry-standard solutions employ write slowdowns to prevent write stalls, which can sharply degrade over throughput and significantly increase tail latency.

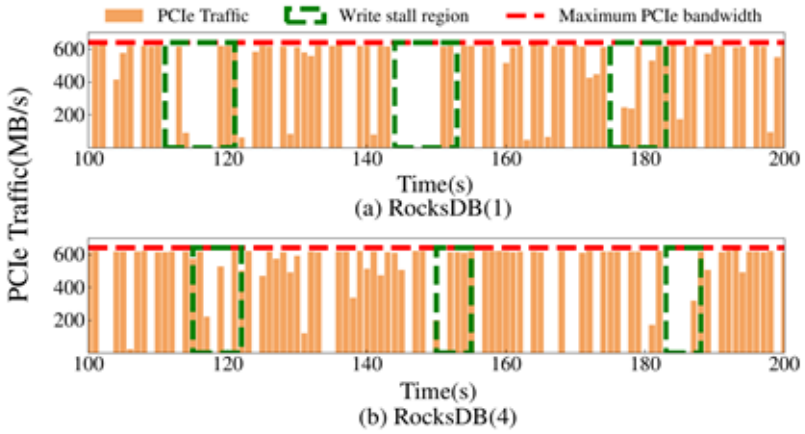


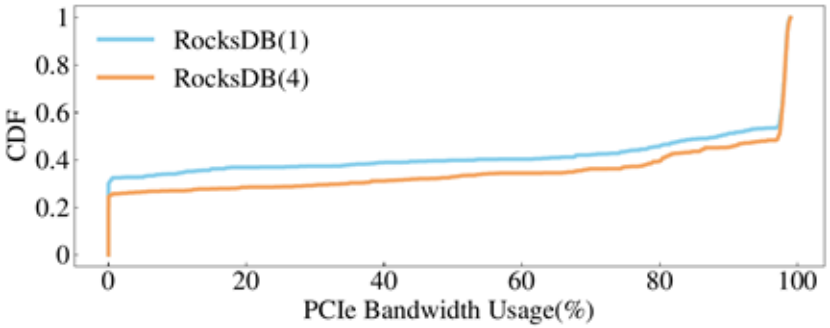
stalls...



Observation 2.
Under-utilization of PCIe Bandwidth

- PCIe Traffic drop sharply during a write stall, implying inefficient device resource usage

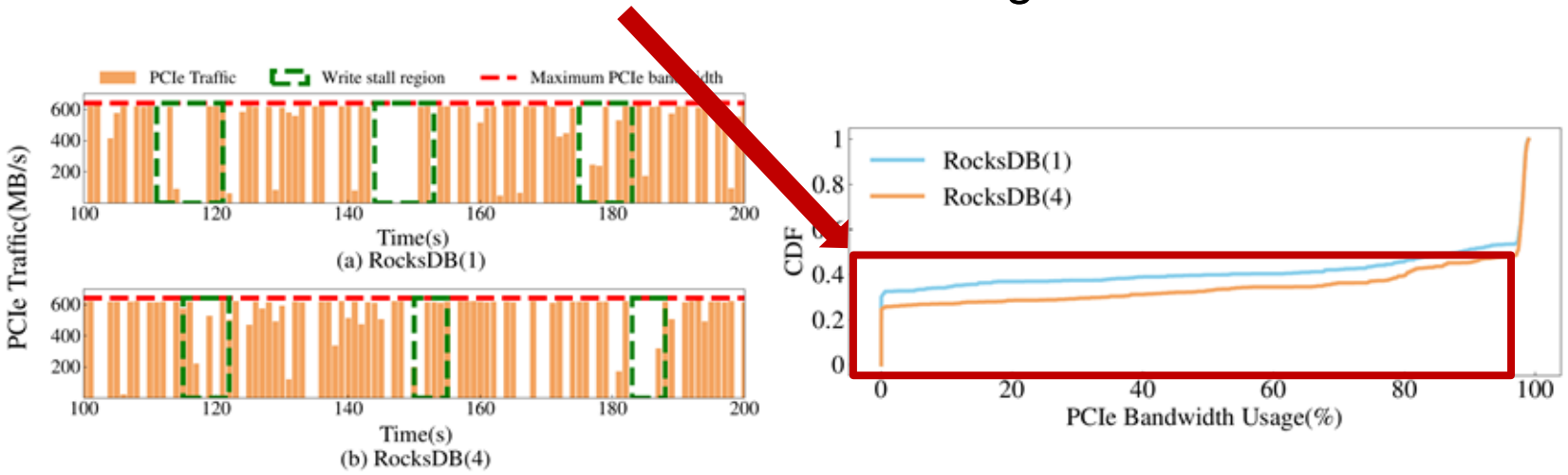




Observation 2.

Under-utilization of PCIe Bandwidth

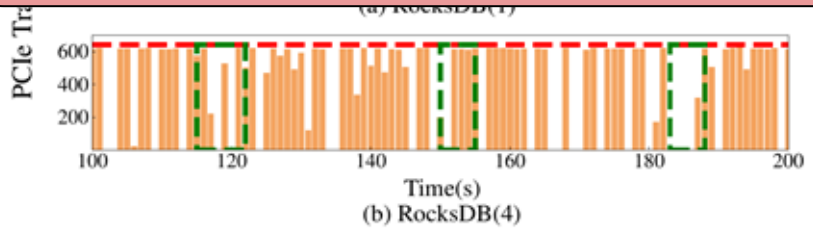
- PCIe Traffic drop sharply during a write stall, implying inefficient device resource usage
  - RocksDB is shown to leave up to **90%** of available PCIe bandwidth around **50%** of the time during a write stall



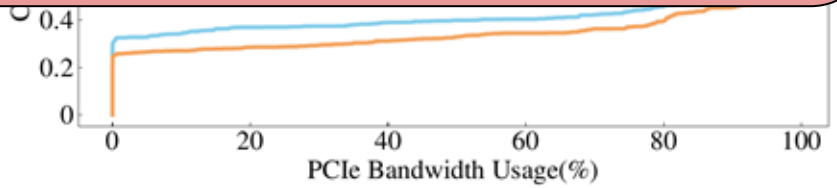
Observation 2.
Under-utilization of PCIe Bandwidth

- PCIe Traffic drop sharply during a write stall, implying inefficient device resource usage.

PCIe bandwidth is under-utilized during write stalls in industry standard LSM-KVS due to the compaction operation blocking device I/O.



(b) RocksDB(4)



The status quo

- **Observation 1.** ultimately leads to the following options for write stalls

Slowdowns	VS	Allowing Write Stalls
• Maintains I/O service at all times • Overall throughput and latency penalty due to said slowdowns		• Overall throughput and latency conserved • Complete interrupts in I/O service as write stalls are allowed to occur

- **Observation 2.** reveals an unexploited resource to help mitigate write stalls and increase performance without sacrificing system resources: underutilized PCIe and device bandwidth during write stalls

The status quo

- **Observation 1.** ultimately leads to the following options for write stalls.

Slowdowns VS Allowing Write Stalls

Can write stalls be mitigated without sacrificing system resources by leveraging underutilized PCIe and device bandwidth during write stalls?

- and increase performance without sacrificing system resources: underutilized PCIe and device bandwidth during write stalls.

Proposed Solution: ***KVAccel***

- ***KVAccel***'s design is based on two key factors: Disaggregation and Aggregation

Disaggregation

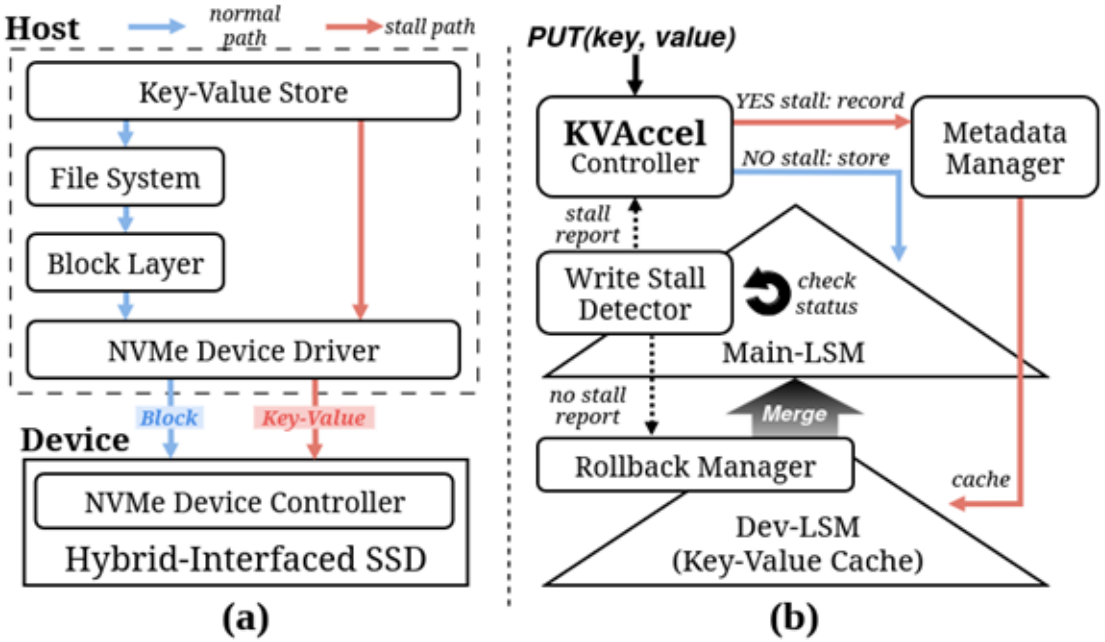
- Division of SSD into hybrid interface (block and key-value) and its required I/O paths
- Maintenance of each interface's separate LSM-Tree

Aggregation

- Manage data from each interface as if it was one database instance
- Unify separate I/O commands and database state with rollback

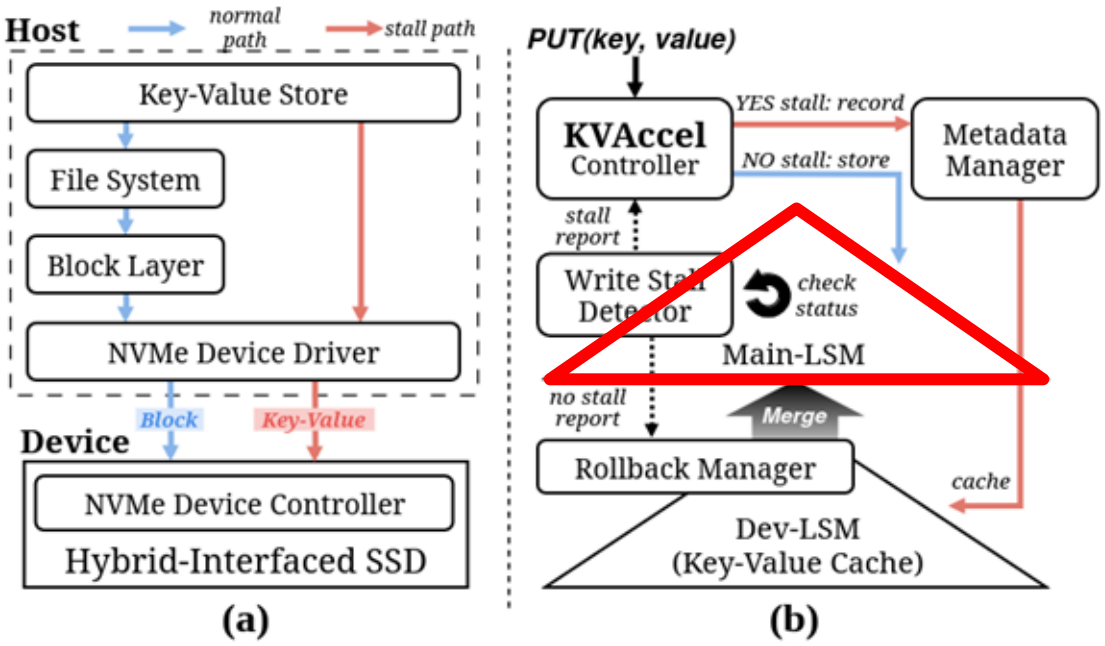
Overview of ***KVAccel***

- Co-Design of Hardware & Software provides 2 I/O paths
- Different I/O paths taken based on the presence of a write stall



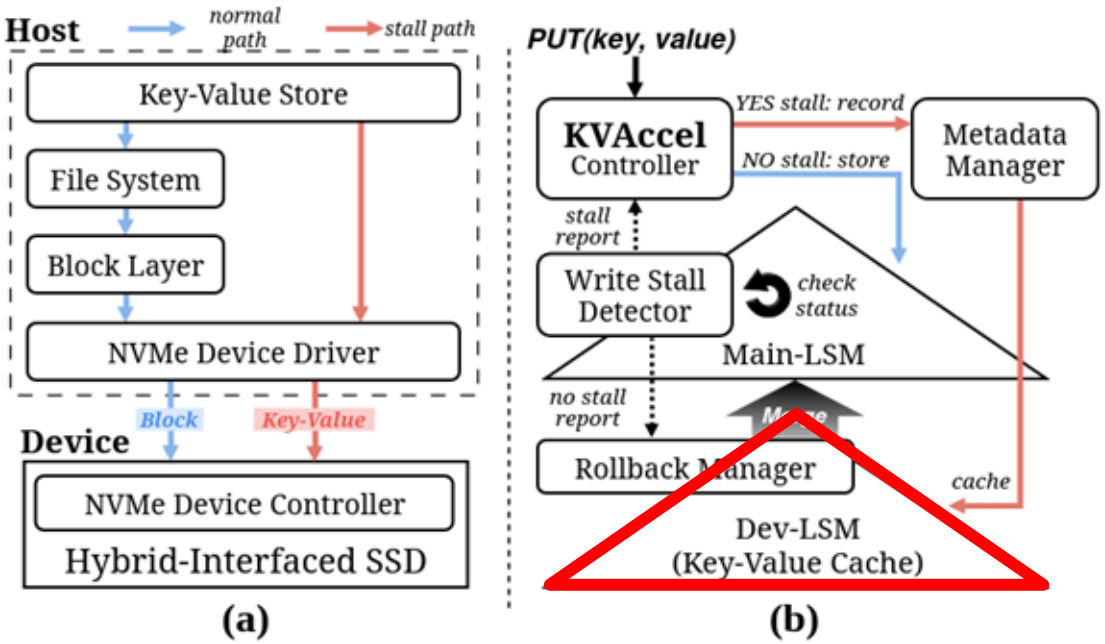
Overview of ***KVAccel***

- Co-Design of Hardware & Software provides 2 I/O paths
- Different I/O paths taken based on the presence of a write stall



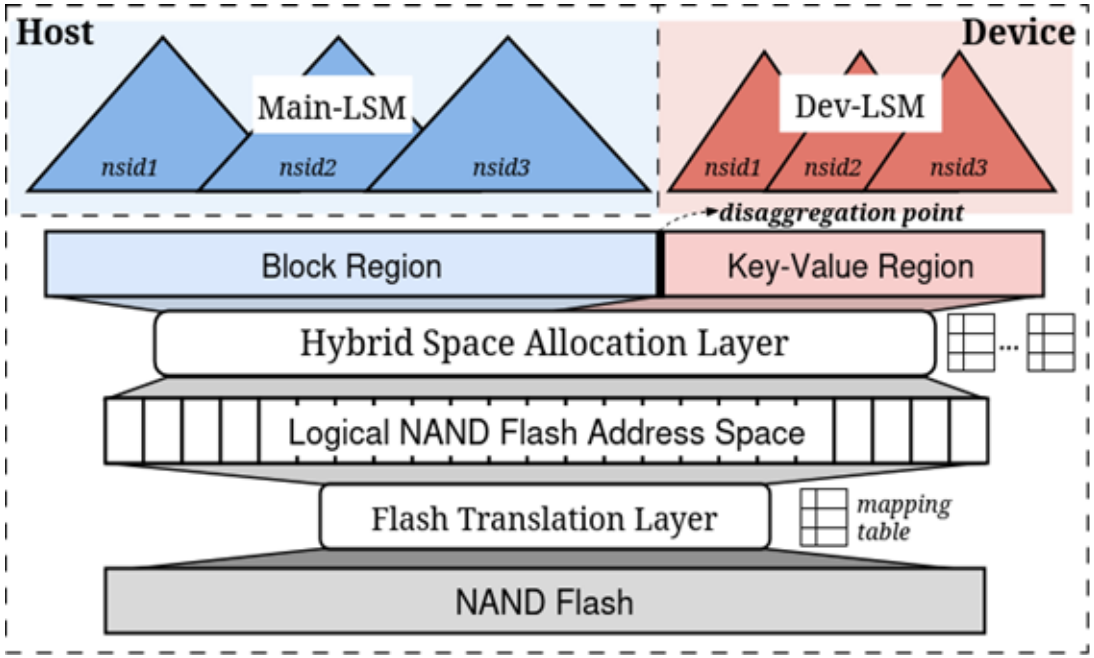
Overview of ***KVAccel***

- Co-Design of Hardware & Software provides 2 I/O paths
- Different I/O paths taken based on the presence of a write stall

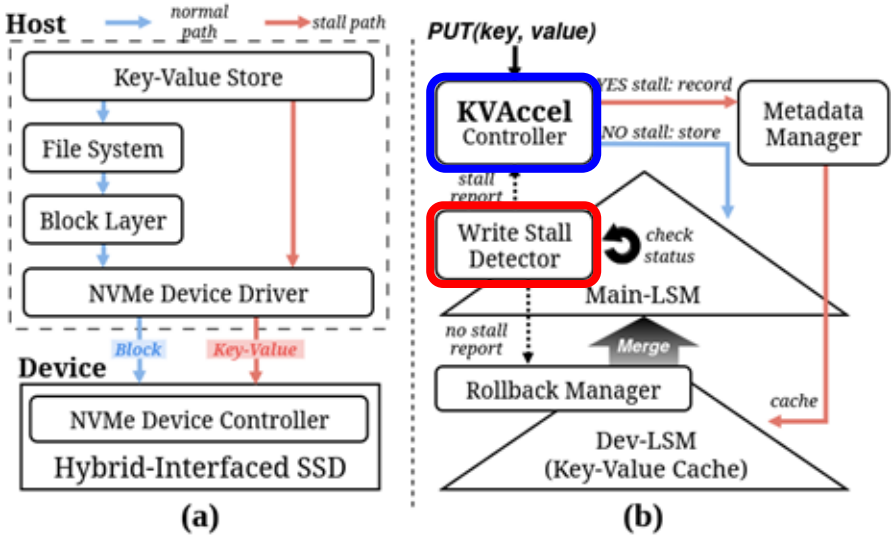


Hybrid Dual-Interface SSD

- Hybrid interface SSD achieved by logical NAND flash address disaggregation via a specified address boundary
  - SSD issues different commands for each interface

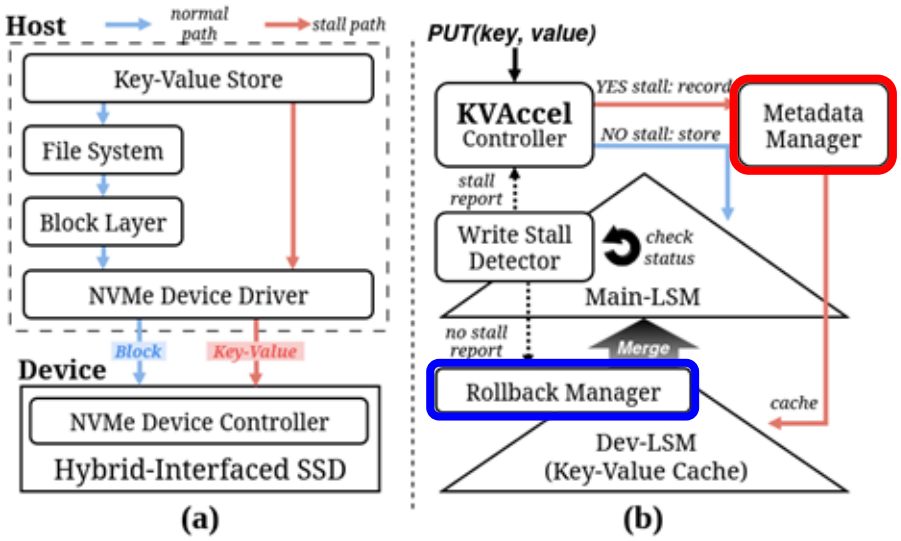


Software Modules(1)



- Detector
  - Detects write stalls checking 3 components
    - # of Level 0 SSTs
    - Memtable size
    - Pending compaction size
- Controller
  - Directs I/O commands to the correct interface based on the Detector's output.

Software Modules(2)



- Metadata Manager
  - Keeps track of KV pairs located in Dev-LSM via a hash table for membership testing
- Rollback Manager
  - Initiates and performs the rollback operation based on the rollback scheduling policy and the Detector's output

Rollback Operation: Scheduling

- Rollback refers to return the KV pairs in Dev-LSM back to Main-LSM into one LSM-KVS instance
- Rollback operation can be scheduled *eagerly* or *lazily* based on workload characteristics

Eager Rollback

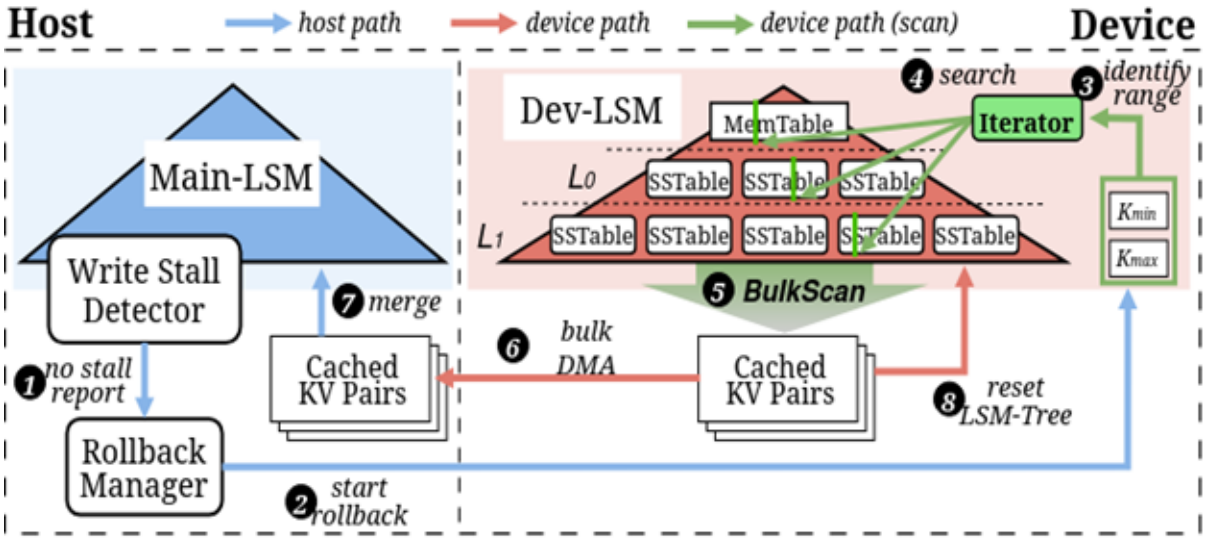
- Perform rollback as soon as there are enough resources available (by using L_0 file count threshold)
- Ideal for a read orientated workload to avoid slow Dev-LSM read operations

Lazy Rollback

- Delay rollback until the current write workload is completely finished
- Ideal for a write intensive workload to lower interference of rollback with write operations

Rollback Operation

- To accelerate rollback, KV pairs are read in bulk using a range scan operation
- Iterator reads Dev-LSM in its entirety and serializes the KV pairs
- KV pairs are then sent to the host by performing DMA multiple times



Evaluation Setup

- Testbed:

KV-SSD on Cosmos+ OpenSSD Platform[7]

TABLE I: Specifications of the OpenSSD platform.

SoC	Xilinx Zynq-7000 with ARM Cortex-A9 Core
NAND Module	1TB, 4 Channel & 8 Way
Interconnect	PCIe Gen2 ×8 End-Points

TABLE II: Specifications of the host system.

CPU	Intel(R) Xeon(R) Gold 6226R CPU @ 2.90GHz (32 cores), CPU usage limited to 8 cores.
Memory	384GB DDR4
OS	Ubuntu 22.04.4, Linux Kernel 6.6.31

[7]: Cosmos+ OpenSSD Platform: http://www.openssd-project.org/platforms/cosmospl/

LSM-KVS and Benchmark Configurations

TABLE III: LSM-KVS configurations. For all figures, the numbers next to each LSM-KVS refer to compaction thread count. For KVACCEL, the settings refer to the Main-LSM.

LSM-KVS	Compaction Threads (n)	MT Size
KVACCEL(n)	1	128 MB
	2	
	4	
RocksDB(n)	1	
	2	
	4	
ADOC(n)	1	
	2	
	4	

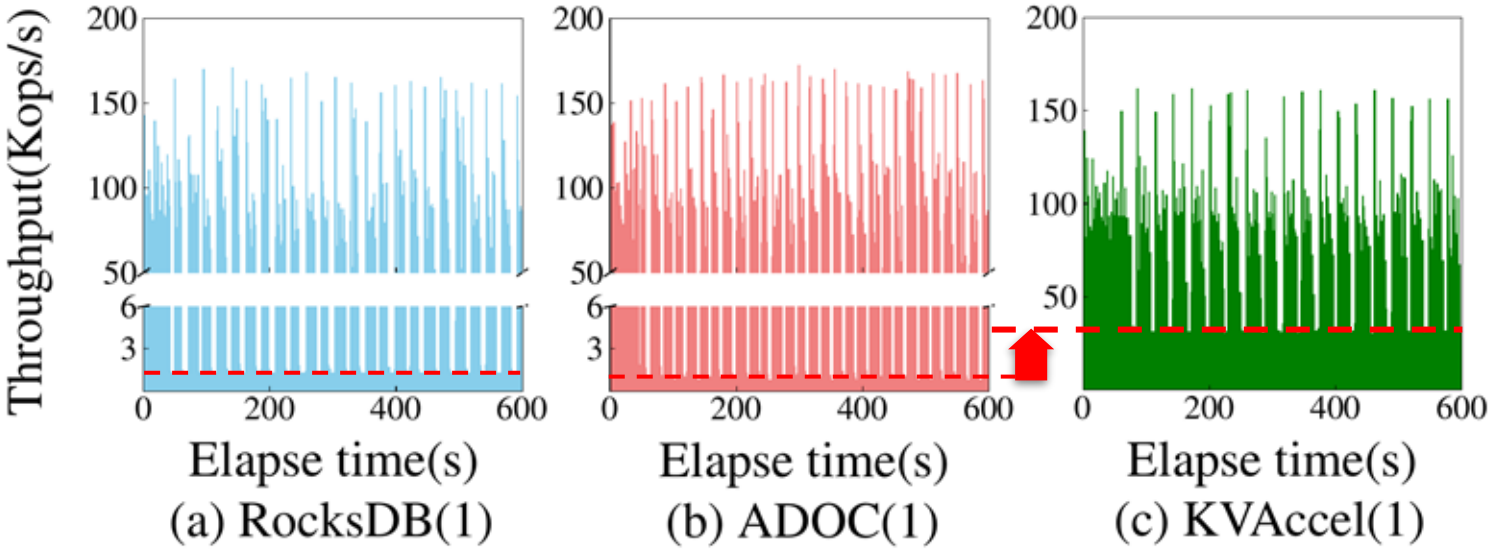
TABLE IV: *db_bench*[8] workload configurations. Each benchmark was run with a 4 B key and 4 KB value size. Workload A,B,C were run for 600 seconds, and Workload D performed 60K read operations.

Name	Type	Characteristics	Notes (write/read ratio)
A	fillrandom	1 write thread	No write limit
B	readwhilewriting	1 write thread + 1 read thread	9:1
C			8:2
D	seekrandom	1 range query thread (Seek + 1024 Next)	Run after initial 20GB fillrandom

[8]: Facebook, "DB Bench" https://github.com/facebook/rocksdb/wiki/

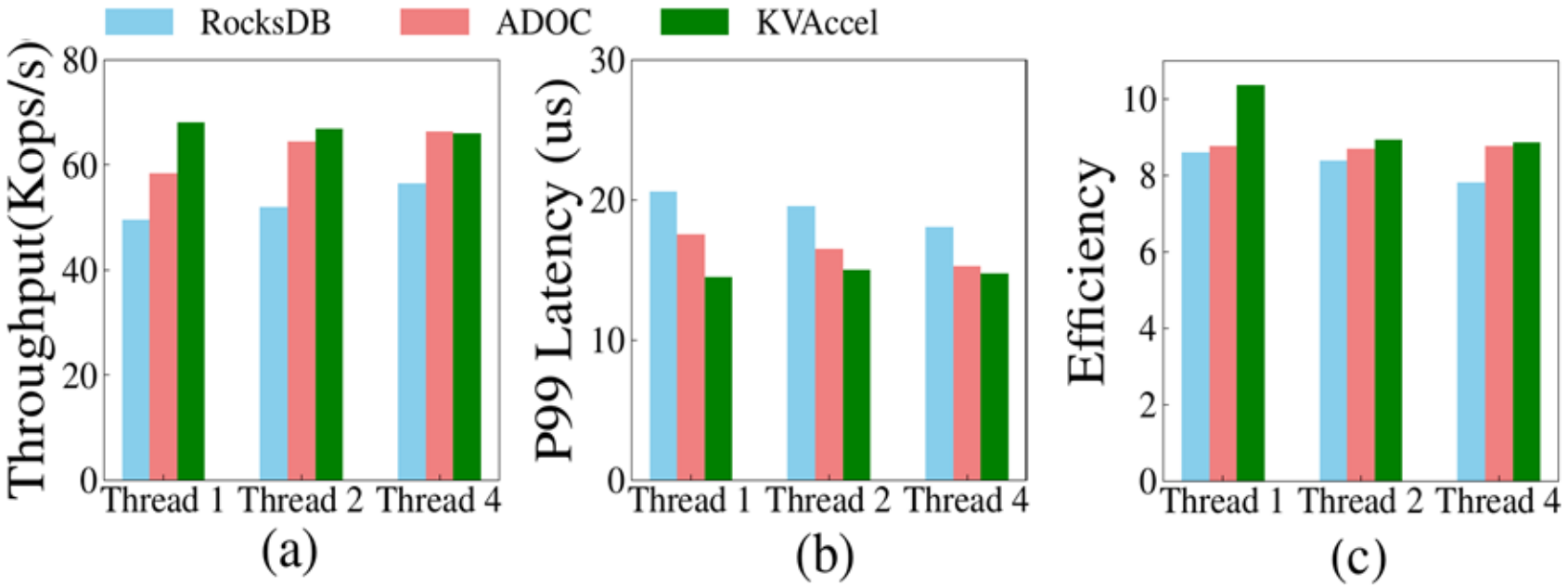
Write Stall Avoidance

- Throughput minimum values greatly increased, as ***KVAccel*** is designed to allow as much throughput as the SSD and system allows without slowdowns



Performance Evaluation

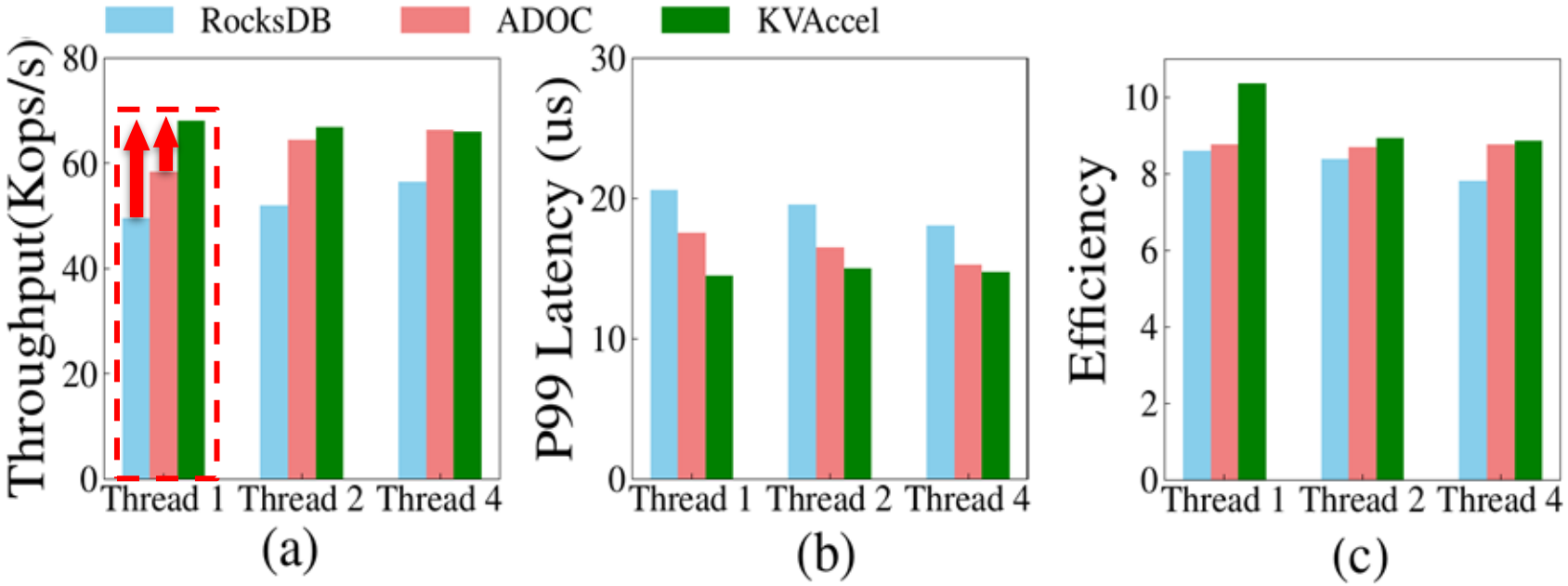
- (a) Throughput, (b) P99 Latency, (c) Efficiency



Performance Evaluation

(a) Throughput

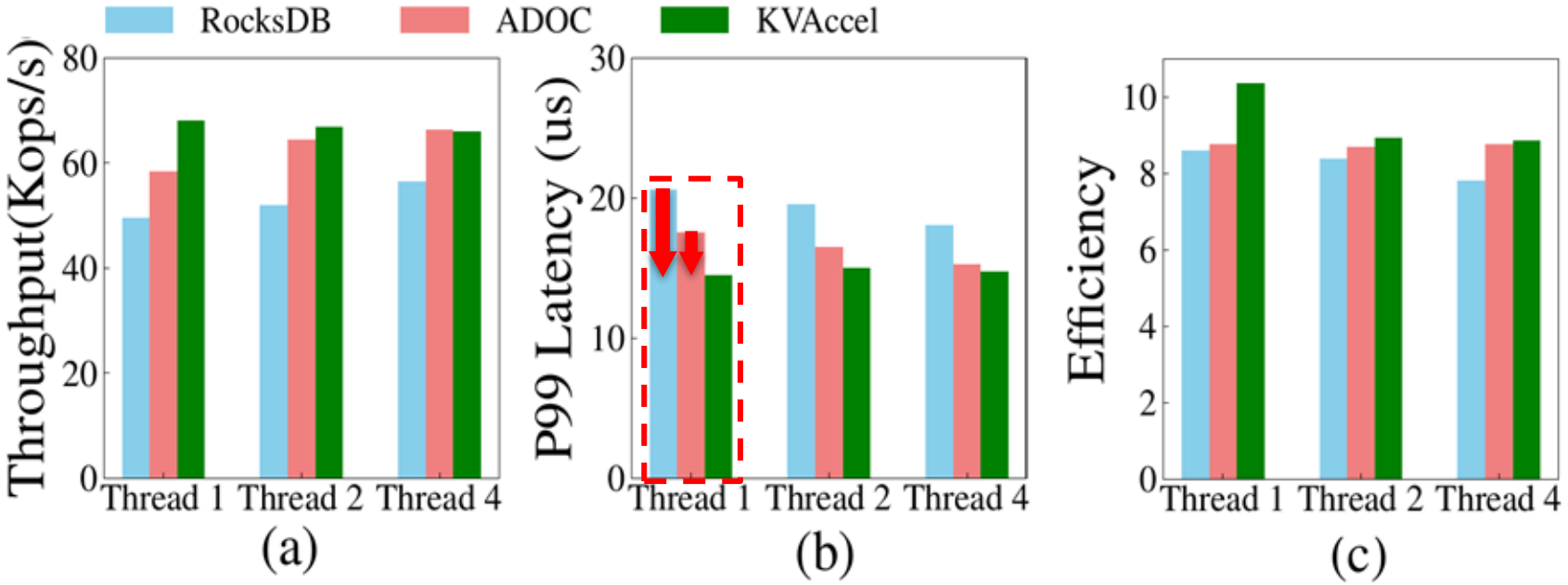
- ***KVAccel*** shows at most a **37%** and **17%** improvement over than RocksDB and ADOC, respectively



Performance Evaluation

(b) Throughput

- Maximum of **30%** and **20%** decrease in latency was also observed between ***KVAccel*** and RocksDB, ADOC, respectively

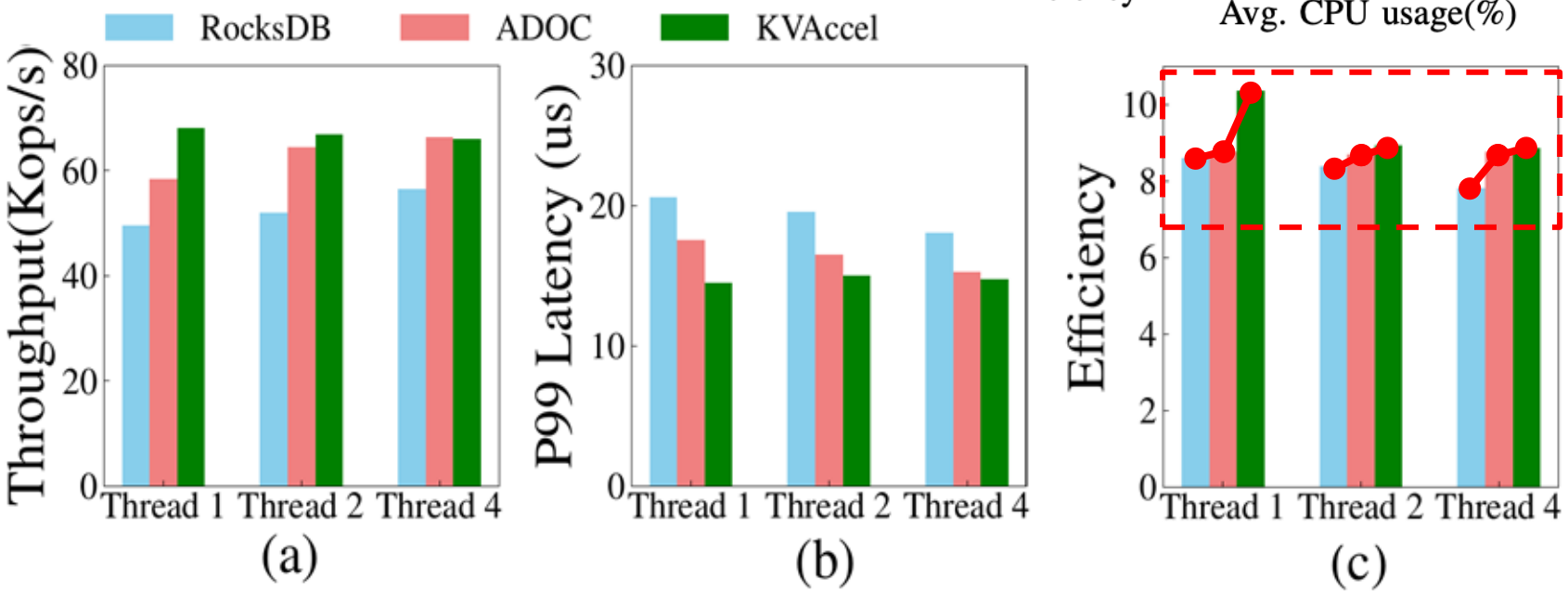


Performance Evaluation

(c) Efficiency

- ***KVAccel*** maintains the better efficiencies in host machine's resources between all LSM-KVS compared

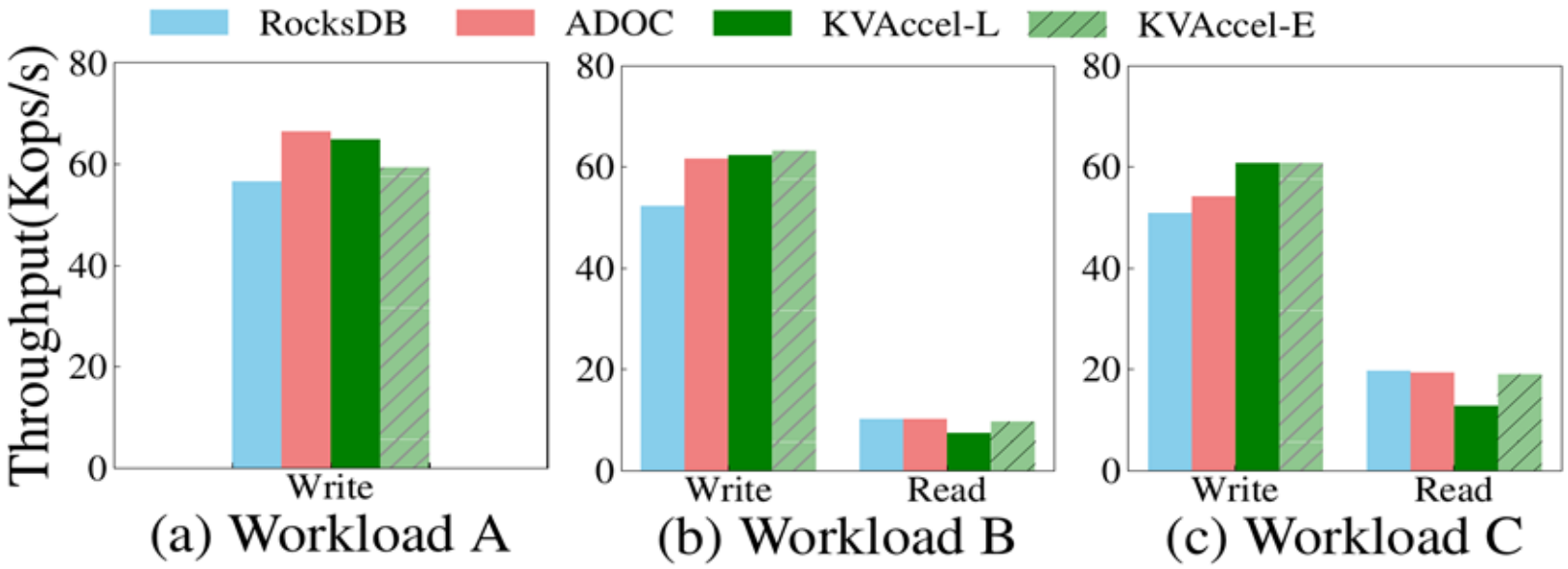
$$\text{Efficiency} = \frac{\text{Avg. Throughput(MB/s)}}{\text{Avg. CPU usage(\%)}}$$



Rollback Policies Evaluation

Eager vs Lazy Rollback analysis

- From (b) and (c), we observe that it still outperforms RocksDB and ADOC under read-oriented workloads

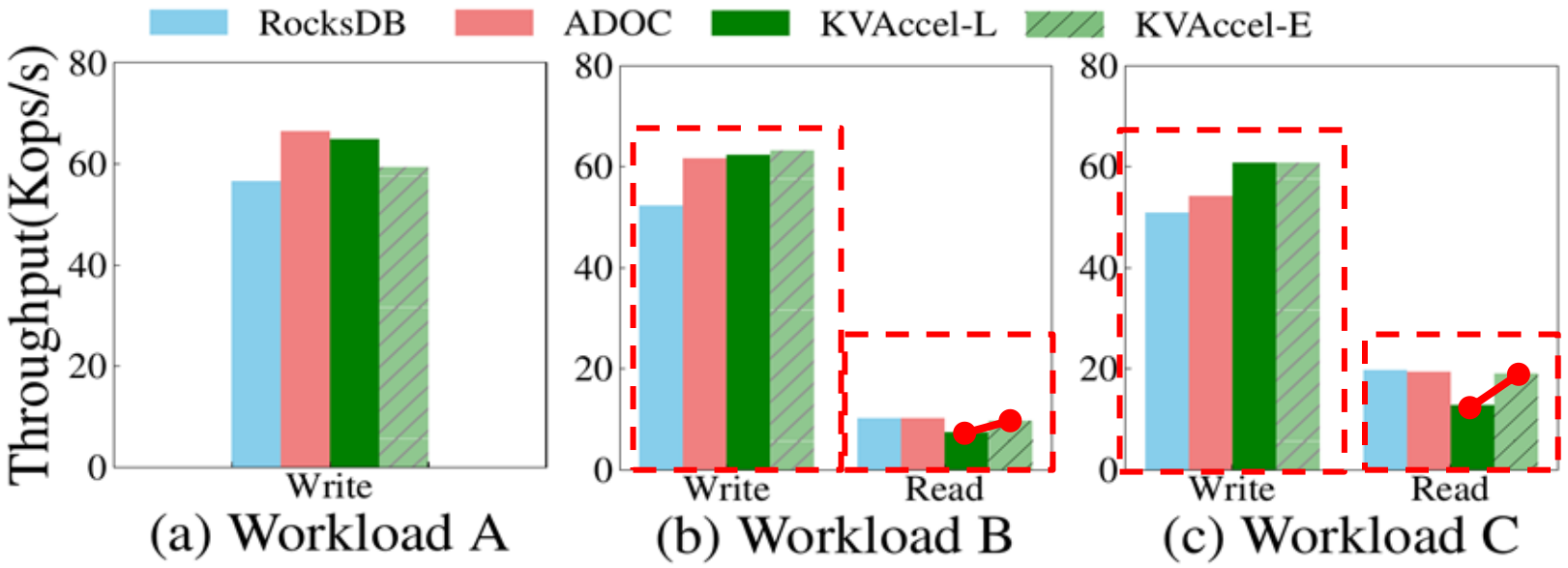


W:R=10:0 **W:R=9:1** **W:R=8:2**

Rollback Policies Evaluation

Eager vs Lazy Rollback analysis

- As the read ratio increases, Eager Rollback becomes increasingly advantageous



W:R=10:0 **W:R=9:1** **W:R=8:2**

Conclusion

- Prior work addresses write stalls to a limited extent
  - Hardware and software are treated in isolation

- ***KVAccel*** achieved a 17% improvement in throughput and a 20% reduction in latency compared to ADOC.

- ***KVAccel*** demonstrates the effectiveness of hardware-software co-design
  - Alleviates write stalls by utilizing:
    - Under-used PCIe bandwidth
    - Computational capabilities within SSDs